



ZLang HDL Community

Language Reference

Compiler version 0.1.0a20

Installation, language semantics, tooling, RTL generation, and verification

Apache-2.0 Community release reference

Contents

1	Installation, WSL2, and external tools	4
1.1	Recommended installation with uv	4
1.2	Conventional venv and pip alternative	4
1.3	Windows through WSL2	5
1.4	Check and run ZLang	5
1.5	Choose which external tools you need	6
1.6	Recommended: OSS CAD Suite	6
1.7	Alternative: install components separately	7
1.8	Verify the installation	7
1.9	Verification architecture and troubleshooting	7
2	Native simulation	7
2.1	Native simulation limits	8
3	Getting started	8
3.1	Install from a source checkout	8
3.2	A first module	9
3.3	Check named verification goals	9
3.4	Source files	10
3.5	Reading common notation	10
3.6	Where to continue	10
4	Types, numerics, and packed representation	11
4.1	Characters, fixed strings, and tuples	11
4.2	Nominal enums	11
4.3	Nominal tagged unions	12
4.4	Scalar families	12
4.5	Integer width rules	13
4.6	Fixed point	13
4.7	Vectors and structs	14
4.8	Slicing, concatenation, and representation	15
4.9	Boundaries	17
5	Expressions, functions, and generics	17
5.1	Operators and selection	17
5.2	Aggregate value ergonomics	18
5.3	Pure functions	18
5.4	Compile-time functions and selection	20
5.5	Functional datapath	20
5.6	Typed bit and collection composition	21
5.7	Generic structs, functions, and operators	22
6	Sequential state, rules, pipelines, and storage	23
6.1	Clock and reset	23
6.2	Registers and next state	24
6.3	Delays and fixed pipelines	25
6.4	Exact module timing contracts	25
6.5	Guarded atomic actions	26
6.6	FIFOs	27
6.7	Writable memories	27
6.8	Initialized synchronous ROMs	31
6.9	Stateful hierarchy	31
7	Physical clocks and resets	32
7.1	Recommended asynchronous reset	32
7.2	Backend lowering	33
7.3	Formal applicability	33
7.4	Boundaries	33

8	Hierarchy, protocols, and explicit CDC	34
8.1	Modules and instances	34
8.2	Clock and reset boundary	35
8.3	Public top ABI	36
8.4	Ready/valid	36
8.5	Credit and request/response	36
8.6	Aggregate protocols and the standard library	37
8.7	Packets, virtual channels, and arbitration	37
8.8	CSR blocks	37
8.9	Clock-domain crossings	39
9	Named module interfaces	39
9.1	Exact conformance	40
9.2	Aggregate protocol signatures	40
9.3	Selection and backend identity	41
9.4	Boundaries	41
10	Tagged unions	41
10.1	Type and layout	41
10.2	Construction and matching	42
10.3	Deliberate boundaries	42
11	Parameterized aggregate protocols	42
12	Standard buses	43
12.1	Bounded AXI burst subset	44
12.2	AXI4-Lite, APB, and RegBus	44
13	Standard library	44
13.1	Fixed-point math	45
13.2	Complex values, streams, storage, and coding	46
13.3	Target and architecture descriptions	47
14	Projects and dependencies	47
14.1	Project manifest	47
14.2	Lock update	48
14.3	Offline compilation	49
14.4	Identity and artifacts	49
14.5	Deliberate boundaries	49
15	Implementation intent and formal verification	49
15.1	Semantic and selected designs	50
15.2	The <code>implement</code> form	50
15.3	Latency, II, and elastic pipelines	51
15.4	Verification goals and contracts	52
15.5	Running verification	53
15.6	Verification boundaries	53
15.7	Reports	53
16	Optimization model	53
17	Implementation profiles	54
17.1	Region selection	54
17.2	Conflicts	54
17.3	Reading the selection report	54
18	Target platforms	55
18.1	Manual architecture selection	55
18.2	Target boundaries	55
19	Target resources	55
19.1	DSP48E1	56

19.2	Memories and asynchronous FIFOs	56
19.3	Intel and ASIC status	56
19.4	Clock resources	56
20	Target-aware implementation selection	56
20.1	User contract	56
20.2	Evidence and exact latency	57
20.3	Boundaries	57
21	Platform constraints	57
21.1	Clock constraints	57
22	Direct-SV	57
22.1	Supported composition subset	58
22.2	Simulation-only architectural state access	59
22.3	Formal boundary	59
22.4	Explicitly unsupported	59
22.5	Examples	59
23	Editor and tooling integration	60
23.1	Diagnostic edits	60
23.2	Query behavior	60
24	Language Server Protocol and VS Code	60
24.1	Supported requests and notifications	61
24.2	Cache configuration	61
24.3	Not implemented	61
25	Structured diagnostics	61
26	Generated RTL source maps	62
26.1	Supported coverage	62
27	Generated artifact bundles	62
28	Build manifests and evidence	63
28.1	Evidence meanings	63
28.2	Bundle generation and proof execution	63
28.3	Manifest and report commands	63
28.4	Limitations	64
29	Source and product identity	64
30	Language support matrix	64
30.1	Language and execution	64
30.2	Formal coverage	66
31	Known limitations	66
31.1	Language and backend boundaries	66
31.2	Verification boundaries	67
A	ZLang HDL concise source-authoring reference	68
A.1	Non-negotiable rules	68
A.2	Smallest valid modules	68
A.3	Types, literals, and exact conversions	69
A.4	Aggregates and functions	69
A.5	State, hierarchy, and protocols	69
A.6	Functional datapath and implementation intent	70
A.7	Verification	70
A.8	Commands and completion check	70

This is the complete user-facing reference for ZLang HDL Community. ZLang separates hardware semantics from implementation intent: source describes exact functional and timing behavior, while implementation constraints guide the selection of legal implementations. Direct SystemVerilog (Direct-SV) is the supported production RTL path. Unsupported constructs are diagnosed explicitly; ZLang does not silently emit partial or guessed hardware.

This reference covers installation, language semantics, compilation, editor integration, implementation planning, generated RTL, and verification. The companion [quick reference](#) is optimized for daily authoring. The PDF edition appends that quick reference to this document.

1 Installation, WSL2, and external tools

ZLang HDL supports CPython $\geq 3.12, < 3.13$ on Linux x86-64. Native-simulation wheels are supplied for Linux x86-64 (including WSL2). macOS native wheels are not part of this release. You do not need to find a distribution package for that exact Python runtime: the recommended `uv` workflow can download and manage it independently of the system Python. Parsing, semantic checking and Direct-SV generation need only the Python package. Verilator, Yosys, SymbiYosys and a solver are external programs used only when their corresponding lint, synthesis or formal flow is requested.

The versions used to build this release are recorded in `release/status.json`. Other external-tool versions may work; validate them for the flow you intend to use.

1.1 Recommended installation with uv

Install `uv` using its official platform instructions. Confirm that the executable is visible in the same terminal that will run ZLang:

```
uv --version
```

For ordinary use, install the wheel downloaded from the matching GitHub release as an isolated command-line tool. `uv` provisions the requested interpreter if it is not already present:

```
uv tool install --python '>=3.12,<3.13' \
  --with /path/to/zlang_native_sim-VERSION-cp312-abi3-PLATFORM.whl \
  /path/to/zlang_hdl-VERSION-py3-none-any.whl
zlang --version
zlang lock --version
zlang verify --version
zlang lsp --version
```

If `uv` reports that its tool directory is not on `PATH`, run `uv tool update-shell`, then start a new shell. `uv tool dir --bin` prints the command directory for manual `PATH` configuration. To replace an installation made from a downloaded wheel, repeat `uv tool install --force ...` with the new wheel.

For an editable Git checkout, keep the environment local to the repository:

```
git clone https://github.com/postoroniy/zlang-hdl.git
cd zlang-hdl
uv venv --python '>=3.12,<3.13'
uv pip install -e '[test]'
source .venv/bin/activate
zlang --version
```

The commands below use `zlang` directly: it is on `PATH` after `uv tool install`, or after activating the checkout's virtual environment with `source .venv/bin/activate` in a Bash/Zsh terminal. Alternatively, in the checkout use `uv run --no-sync zlang ...` without activating it. Open a new terminal and activate the environment again when returning to the checkout.

1.2 Conventional venv and pip alternative

If a compatible Python is already available, use whatever executable name the host provides; the example deliberately avoids assuming a version-suffixed command:

```
python3 -c 'import sys; assert (3, 12) <= sys.version_info < (3, 13), sys.version'
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install \
    /path/to/zlang_native_sim-VERSION-cp312-abi3-PLATFORM.whl \
    /path/to/zlang_hdl-VERSION-py3-none-any.whl
zlang --version
```

For an editable checkout, replace the two-wheel installation command with:

```
python -m pip install -e '.[test]'
```

This fallback needs a compatible interpreter and `venv` support from the host. Do not replace or upgrade the operating system's own Python merely to install ZLang; use `uv` or an isolated virtual environment.

1.3 Windows through WSL2

The current release is a Linux application, not a native Windows Python package. On Windows 10/11, install WSL2 with an Ubuntu distribution, open the Ubuntu shell, and perform the Linux installation entirely inside it. For example:

```
sudo apt-get update
sudo apt-get install -y curl git build-essential
```

Then install `uv` using its official Linux instructions and follow the `uv` steps above. Keep active projects under the WSL Linux filesystem, for example `~/src/zlang-hdl`, rather than `/mnt/c/...`; this generally gives much better compiler, Git and EDA-tool filesystem performance. Start VS Code through its WSL extension so the editor, `zlang lsp`, Python environment and EDA executables all see the same Linux paths.

Inside a development checkout, these forms are equivalent:

```
source .venv/bin/activate
zlang examples/add.zhl --check
uv run --no-sync zlang examples/add.zhl --check
```

`uv run --no-sync` uses the already-created project environment without changing dependencies. In scripts, either activate the environment first or use `uv run --no-sync`; do not depend on a user's interactive shell state.

1.4 Check and run ZLang

Check the installed commands:

```
zlang --version
zlang lock --version
zlang verify --version
zlang lsp --version
```

`zlang lsp` is a JSON-RPC stdio server and is normally started by the bundled VS Code extension; it is not an interactive shell command. Point the extension at the environment's executable when automatic discovery is not appropriate:

```
{
  "zlang.lsp.path": "/absolute/path/to/.venv/bin/zlang"
}
```

The extension supplies the `lsp` subcommand to the configured `zlang` executable. Manual launches and other clients use the same canonical command.

Check a design and emit production Direct-SV without installing any external EDA program:

```
zlang examples/add.zhl --check
mkdir -p build
```

```
zlang examples/add.zhl --systemverilog build/Add.sv
```

1.5 Choose which external tools you need

Only a compatible Python runtime and the `zlang-hdl` package are mandatory for checking source and emitting SystemVerilog. `uv` is an optional installer: it can obtain that Python runtime, but ZLang does not need `uv` after installation. Install other tools only for the workflows you actually use:

Workflow	Additional requirement	If it is missing
<code>zlang SOURCE --check</code> and SystemVerilog emission	None beyond Python and <code>zlang-hdl</code>	Neither Verilator nor Yosys is needed; a missing compiler package or incompatible Python prevents <code>zlang</code> from starting.
<code>zlang sim</code> (native engine)	Matching <code>zlang-native-sim</code> wheel for the host	Simulation fails explicitly; compiler checking and RTL emission remain available.
VS Code diagnostics and navigation	Bundled extension plus <code>zlang lsp</code> from the base package	Compiler CLI still works, but editor integration does not start.
Strict RTL lint or Verilator execution	<code>verilator</code>	RTL can still be emitted; lint and Verilator simulation cannot run.
Generic synthesis	<code>yosys</code>	RTL can still be emitted; synthesis cannot run.
Formal BMC, proof, or cover	<code>yosys</code> , <code>sby</code> , <code>yosys-smtbmc</code> , and the <code>z3</code> executable	Checking and RTL emission still work; requested formal jobs cannot complete.
Icarus-based RTL simulation flow	<code>iverilog</code> and <code>vpv</code>	Only that selected flow is unavailable.
<code>zlang lock update</code> with Git dependencies	<code>git</code> and access to the pinned revision during update	Path-only projects and locked offline compilation still work; fetching a Git dependency does not.

`yosys-smtbmc` is the Yosys SMT model-checking driver. It is the executable sometimes informally shortened to “BMC”; there is no ZLang dependency named `bmct`. ZLang's recorded formal route invokes the external `z3` executable through this driver. Installing only the Python `z3-solver` module is therefore not a substitute for putting `z3` on `PATH`.

1.6 Recommended: OSS CAD Suite

The simplest reproducible route is the official YosysHQ OSS CAD Suite. Its release archive provides Yosys, SymbiYosys, solvers and related open-source EDA tools together. Download the archive for the host platform from the [OSS CAD Suite releases](#), extract it, then either source its environment file for each shell:

```
source /absolute/path/to/oss-cad-suite/environment
```

or add its `bin` directory to `PATH`:

```
export PATH="/absolute/path/to/oss-cad-suite/bin:$PATH"
```

Use an absolute installation path. Do not copy individual binaries out of the suite because Yosys and SymbiYosys also use adjacent data and support files. See the upstream OSS CAD Suite release page linked above for supported archives and platform details.

Install Verilator separately when it is not supplied by the selected suite. A distribution package is convenient:

```
sudo apt-get update
sudo apt-get install verilator
```

Distribution versions vary. Follow the upstream [Verilator installation guide](#) when a newer distribution package or a source build is required.

1.7 Alternative: install components separately

Component installation is useful for distribution packaging or when a pinned tool is maintained independently:

- install Yosys using the upstream [Yosys build/install instructions](#);
- install SymbiYosys using the upstream [SymbiYosys documentation](#);
- install the z3 command from the official [Z3 releases](#) or a suitable distribution package;
- install Verilator using its official guide linked above;
- install Icarus Verilog from the host distribution when the selected RTL simulation flow requires iverilog/vvp.

Keep all selected tool bin directories on the same PATH used to start ZLang, the terminal, an automated build, or VS Code. Mixed host/container installations are a common reason for a tool being visible interactively but unavailable to the compiler.

1.8 Verify the installation

```
verilator --version
yosys -V
sby --version
command -v yosys-smtbmc
z3 --version
iverilog -V
```

The external-tool versions used for this release are recorded in the [release status](#) under `eda_toolchain`; the supported Python range is recorded under `platform.python`. These are evidence for that release, not universal tool-version requirements. Other versions may work; validate them for the flow you intend to use. Finding an executable on PATH is not, by itself, evidence that a formal or synthesis result is valid.

Run a small formal check:

```
zlang examples/verification/bounded_counter.zhl \
--verify --verify-require proven \
--formal-depth 16 --formal-timeout 45 \
--verification-bundle build/verify/counter \
--verification-work-dir build/verify-work/counter \
--verification-report build/verify/counter-result.txt
```

Expected exit status is zero. The capacity assertion should be proven, the `reaches_capacity` cover should be witnessed, and `exceeds_capacity` should be bounded_unreached. A bounded cover miss is not a proof of unreachability. See the [runnable formal examples](#) for a deliberate counterexample, scoped assumptions, immutable bundle replay and ready/valid checking.

1.9 Verification architecture and troubleshooting

Formal execution is opt-in. `--check`, normal compilation and SystemVerilog emission do not invoke a solver. `bounded_pass` is bounded evidence and is never reported as proven; unavailable tools or bindings remain explicit `unknown/skipped` according to the requested policy.

If a tool is not found, run the version commands above in the same environment that starts ZLang. If a proof is unknown, inspect the retained report and solver logs under `--verification-work-dir`; increasing depth or timeout does not turn an unsupported binding or reset model into an executable property.

2 Native simulation

ZLang provides native simulation through the matching `zlang-native-sim` package for Linux x86-64, including WSL2. The simulator executes the same typed design semantics used for compilation and supports persistent instances, clock events, VCD traces, and optional cycle-by-cycle comparison with generated RTL. Parsing, checking, and Direct-SV generation remain available without this package; `zlang sim` requires it.

```
from zlang import sim

program = sim.compile("examples/counter.zhl", top="Counter", engine="native")
```

```
instance = program.create()
instance.tick("clk")
print(instance.get("y"))
instance.close()
```

The command-line interface supports the same explicit engine choice:

```
zlang sim examples/counter.zhl --top Counter --engine native \
  --clock clk --cycles 100 --json
zlang sim examples/multi_clock_stateful.zhl --top MultiClockStateful \
  --engine native --events events.jsonl --trace build/trace.vcd
zlang sim examples/add.zhl --top Add --set a=200 --set b=55 \
  --compare-with iverilog --compare-artifacts build/sim-compare --json
```

`--compare-with iverilog` and `--compare-with verilator` emit the normal direct SystemVerilog artifact, execute it in the selected external simulator, and compare every physical output after every event. A mismatch, X/Z value, missing sample, tool failure, or timeout fails closed. Comparison artifacts are retained only when an empty `--compare-artifacts` directory is supplied. Sequential RTL must receive the same explicit reset events needed by physical hardware; use `--events` when comparing a resettable design.

One persistent instance is not thread-safe; independent instances may run concurrently. `eval()` changes no architectural state. `edge_many()` computes all next-state values from one pre-edge snapshot and commits atomically. Synchronous resets act on their domain edge, while supported asynchronous assertion acts immediately. Values are exact-width and are never silently truncated.

2.1 Native simulation limits

The following are implementation limits of this compiler release's native simulator, not ZLang language semantics. Native simulation accepts at most:

- 32,768 compiled simulation operations;
- a cumulative packed-value work budget of 32,768 machine words;
- 16 MiB of compiled simulation data;
- packed values up to 8,192 bits;
- arithmetic operands up to 512 bits; and
- memory elements up to 8,192 bits.

Generated functional work is also bounded to 8 nested regions, 1,000,000 total iterations, and an estimated runtime-work budget of 8,000,000 operations. Compilation fails with a diagnostic when a limit is exceeded.

There is no silent fallback from native execution to an RTL simulator. `native` is the supported simulation engine; other engine names are rejected. Unsupported external stateful/protocol models remain unsupported rather than acquiring simulator-specific behavior.

3 Getting started

The representative executable language tour is `examples/all_syntax.zhl`. It intentionally does not enumerate every legal composition or backend boundary. The [syntax support matrix](#) distinguishes supported, bounded, and unsupported forms.

Physical ZLang HDL source files use the canonical `.zhl` suffix and MIME type `text/x-zlang-hdl`. Logical imports remain extension-independent. The former `.zl` spelling is intentionally rejected so these sources cannot be confused with the unrelated language that already owns that extension.

3.1 Install from a source checkout

For development from a checkout, follow the source-checkout installation steps in [Chapter 1](#). That section covers the supported Python environment, WSL2, optional EDA tools, and both `uv` and conventional `venv` workflows.

Check a source file without creating backend artifacts:

```
zlang examples/all_syntax.zhl --check
```

Without `--top`, `--check` validates every declared module. With `--top NAME`, it validates only that elaboration root. This is a semantic-only demand: it does not run implementation planning, formal execution, report rendering, or the SystemVerilog backend.

3.2 A first module

```
module Add {
    in  a : u8
    in  b : u8
    out y : u9

    y = a + b
}
```

Hardware assignments are concurrent. Source order does not turn `=` into software-style sequencing. `=` drives a combinational value; `<=` schedules a register's next value at its clock edge.

Generate Direct-SV and lint it:

```
zlang examples/extended_add.zhl \
    --systemverilog build/ExtendedAdd.sv
verilator --lint-only --top-module ExtendedAdd build/ExtendedAdd.sv
```

An explicit artifact path keeps stdout empty. Use `--verbose` for success messages on stderr. A bare invocation is rejected; use `--check` for semantic validation without emission.

3.3 Check named verification goals

Clocked modules can carry same-cycle safety and bounded-reachability goals:

```
assert count_within @ clk { count <= DEPTH }
cover reaches_full @ clk { count == DEPTH }
```

Run the applicable verification jobs and source goals:

```
zlang design.zhl --top Top --verify \
    --formal-jobs 4 \
    --verification-report build/verification.txt \
    --verification-work-dir build/verification-work
```

Or publish a hash-validated bundle and replay it without recompiling source:

```
zlang design.zhl --top Top \
    --verification-bundle build/verify
zlang verify build/verify --mode bmc --depth 20 \
    --work-dir build/verify-work
```

`bounded_pass` means only that no counterexample was found through the stated depth. A cover reports `witnessed` or `bounded_unreached`; a cover miss is not a safety failure. A `prove` request runs BMC first and starts proof only when every safety job is `bounded_pass`. Covers run once and are not rerun; an unrelated cover does not block proof, while an unwitnessed feasibility cover makes its dependent safety result vacuous/unknown. Solver configurations, logs, and VCDs are retained in the external work directory; the immutable bundle is not modified. `--formal-jobs` parallelizes independent bundled safety and cover jobs while preserving deterministic report order.

Each verification goal is routed against its own declared clock/reset pair. Goals in two supported synchronous domains can execute independently; an unsupported domain skips only goals in that domain. This does not add a cross-domain temporal property or change the general backend domain boundary.

The formal modes are distinct:

- a non-off `--formal-policy` without `--verify` runs only the equivalence check used during implementation selection;

- `--verify` with policy `off` runs safety verification/source safety and covers;
- `--verification-bundle` publishes safety/cover inputs but does not execute them;
- `--verify` with a non-`off` policy additionally executes the compatible Direct-SV equivalence check between the selected implementation and source semantics.

3.4 Source files

A file may contain imports, type aliases, structs, pure functions, operator overloads, rewrite declarations, target/resource declarations, and one or more modules:

```
import std.math.complex

type Byte = u8

struct Pair<type T> {
  left  : T
  right : T
}

fn widen(x : Byte) -> u9 { extend<9>(x) }

module Example {
  in  x : Byte
  out y : u9 = widen(x)
}
```

Select a top explicitly when a file contains several modules:

```
zlang design.zhl --top Example --check
```

Names are case-sensitive. There are no semicolons. Line comments use `//`. Non-nested block comments use `/* ... */`; both forms are lexical whitespace. Decimal, hexadecimal (`0x`), and binary (`0b/0B`) integer literals may contain valid underscore separators.

3.5 Reading common notation

Form	Meaning
<code>name = expression</code>	Drive a combinational value or resource control.
<code>state <- expression</code>	Schedule next-state at the active edge.
<code>source -> destination</code>	Connect typed endpoints or bind a command.
<code>key => expression</code>	Select a <code>switch</code> /choice arm.
<code>thing @ clk</code>	Associate a declaration with a clock domain.
<code>field @ 7:4</code>	Place a CSR field.
<code>object.field</code>	Select an aggregate, protocol, or resource member.
<code>values[index]</code>	Read a range-proven vector element; the selector may be runtime.
<code>raw[index] for bits<N></code>	Read one LSB-zero packed bit; the selector must resolve at compile time.

ZLang requires exact canonical types at assignment boundaries. It never silently truncates, extends, performs a numeric signed/unsigned conversion, or rescales fixed-point values. Equal-width raw boundaries involving `bits<N>` or flat `vec<N,bit>` are the documented representation-only exception; use `bitcast<T>` explicitly when the boundary is not already typed.

3.6 Where to continue

- [Types and numerics](#)
- [Expressions, functions, and generics](#)
- [Sequential logic, rules, and storage](#)
- [Hierarchy, protocols, and composition](#)

- [Optimization and formal verification](#)
- [Direct-SV, CLI, and tooling](#)
- [Standard library](#)
- [Reproducible projects and dependencies](#)
- [Named module interfaces](#)
- [Direct-SV examples](#)

4 Types, numerics, and packed representation

ZLang types describe hardware representation exactly. Width, signedness, fixed-point scale, vector length, struct identity, and fixed-point overflow policy participate in typing and canonical identity.

4.1 Characters, fixed strings, and tuples

`char` is a source alias for canonical `u8`; it is intentionally not a separate nominal type. `string<N>` is a source alias for the corresponding fixed hardware collection and normalizes to canonical `vec<N,u8>`:

```
letter : char = 'A'
tag : string<4> = "OFDM"
joined = concat("802.", "11a")
```

Literals contain printable ASCII directly and may use `\0`, `\n`, `\r`, `\t`, `\\`, `\'`, `\"`, or an exact `\xNN` byte. Strings have no hidden terminator or length field and cannot be empty. They inherit vector indexing, equality, `length`, `generate/map`, homogeneous concatenation, pure functions, generic specialization, and typed compile-time constant parameters. Registers, FIFOs, synchronous memories, and ROMs store them with the same semantics as `vec<N,u8>`. When packed, the first character is the least-significant byte (`pack("AB") == 0x4241`). Unicode, interpolation, formatting, padding, and dynamic-length strings are not implicit operations.

Tuples are structural ordered aggregates. Tuple types use `(T,U)` and tuple values use the same positional spelling:

```
pair : (u8, bit) = (data, last)
payload = pair[0]
(next_data, next_last) = pair
```

Tuple arity is 2 through 8. Projection is a zero-based integer literal and flat destructuring introduces exhaustive immutable bindings. Nested tuple values are legal, but nested/wildcard patterns, runtime tuple selection, tuple update, arithmetic, and ordering are not. Exact `==/!=` is structural. Component zero occupies the least-significant packed region; `pack/bitcast` require every component to satisfy the ordinary non-enum packing rules.

`char` is indistinguishable from `u8` during overload resolution, and `string<N>` is indistinguishable from `vec<N,u8>`. Tuple identity, by contrast, is the recursive ordered sequence of component types.

4.2 Nominal enums

Enums give control state a nominal hardware type without exposing a numeric encoding in source:

```
enum State { Idle Header Payload Done }

reg state : State = State.Idle
active = state == State.Payload
```

Members use qualified names. Their declaration-order ordinals are encoded in `max(1, ceil_log2(member_count))` bits, so a one-member enum still occupies one bit. The encoding is a stable storage/backend ABI, but enum values are not integers: arithmetic, ordering, resizing, and cross-enum comparison are errors. Equality and inequality require the same nominal enum declaration.

When an external encoding is part of the hardware contract, the representation can instead be declared explicitly:

```
enum WifiRate : bits<3> {
  Continue = 0
  Bpsk6 = 1
  Qpsk12 = 2
}
```

```

    Qam16_24 = 4
}

```

Every member then requires one unique code fitting the exact `bits<W>` backing type. Declaration order still controls exhaustive source selection, while the declared sparse code is the physical representation. Mixing explicit and implicit members is an error. The ordinal form above remains unchanged.

Raw boundaries use total, typed operations rather than enum casts:

```

raw      : bits<3> = enum_encode(rate)
valid    : bit = enum_valid<WifiRate>(raw)
decoded  : WifiRate =
    enum_decode<WifiRate>(raw, WifiRate.Continue)

```

`enum_encode` is lossless. `enum_valid<T>` recognizes exactly the declared codes. `enum_decode<T>` returns its required same-enum fallback for every hole in the encoding, so invalid raw bits never become an invalid nominal value. These operations perform no resize and accept no signed/integer substitute.

Enums may appear in locals, outputs, registers/rules, structs, and vectors. A top-level enum input is rejected because an external invalid bit pattern would violate the nominal value invariant. General `unpack<Enum>` is likewise not supported. Exhaustive selection is described under [operators and selection](#).

4.3 Nominal tagged unions

A tagged union represents one of several named payload shapes without exposing raw tag arithmetic:

```

union Message {
    Idle
    Data { value : u8 }
    Error { code : bits<4> }
}

message : Message = Message.Data { value = input }
result  : u8 = match message {
    Message.Idle => 0
    Message.Data { value } => value
    Message.Error { code } => extend<8>(code)
}

```

The nominal declaration identity is part of the type. The source-order tag is stored in the most-significant bits; fields occupy the maximum-width payload in source order and shorter payloads receive zero low-order padding. A fieldless constructor is written `Message.Idle`. Every pure `match` is exhaustive, names each variant once, and binds exactly the declared fields. See [the complete bounded surface](#).

4.4 Scalar families

Spelling	Meaning
<code>bit</code>	A one-bit control value, distinct from integers and raw bits.
<code>uN, uint<N></code>	An N -bit unsigned integer.
<code>sN, sint<N></code>	An N -bit two's-complement signed integer.
<code>bits<N></code>	An N -bit raw vector without integer arithmetic.
<code>fixed<W,F></code>	Signed fixed point, W total bits and F fractional bits, wrapping target narrowing.
<code>ufixed<W,F></code>	Unsigned fixed point with wrapping target narrowing.
<code>fixed_sat<W,F></code>	Signed fixed point with saturating target narrowing.
<code>ufixed_sat<W,F></code>	Unsigned fixed point with saturating target narrowing.

Widths are positive compile-time integers. Fixed-point types additionally require $0 \leq F < W$. The families are distinct: no implicit signed/unsigned, integer/raw-bit, or integer/fixed conversion is performed.

Aliases disappear during typing:

```

type Address = uint<32>
type Payload = bits<64>

```

Alias cycles, duplicate declarations, and redefinition of built-ins are errors.

4.5 Integer width rules

An unsuffixed integer literal has the smallest exact hardware type implied by its value. Non-negative literals are unsigned; syntactic negative literals are signed two's-complement values:

Literal	Inferred type
0, 1	u1
2, 3	u2
255	u8
-1	s1
-2	s2
-3, -4	s3
-123	s8

Decimal, hexadecimal, and binary spellings with the same value infer the same type; leading zeros do not request storage width. A direct literal at an explicitly typed assignment, field, argument, or return boundary instead uses that target type when the value fits exactly. An out-of-range literal is an error. Context never recursively resizes a compound expression, so `concat(0,1)` and `0 + 1` retain the types derived by their own operators.

Unary `-` applied to a non-literal remains ordinary typed arithmetic. It does not reinterpret an unsigned signal as a signed one.

Expression	Result
<code>uN + uM</code>	<code>u(max(N,M)+1)</code>
<code>sN + sM</code>	<code>s(max(N,M)+1)</code>
<code>uN - uM</code>	<code>u(max(N,M))</code> , modular underflow
<code>sN - sM</code>	<code>s(max(N,M)+1)</code>
same-family <code>N * M</code>	Width <code>N+M</code>
<code>&</code> , <code> </code> , <code>^</code>	Matching family, widest operand width
shifts	The left operand's exact type and width
comparisons	bit

Addition and multiplication preserve carry bits. A left shift discards bits beyond its fixed result width. Signed right shift is arithmetic; unsigned and raw-bit right shift is logical.

Use explicit resizing:

```

wide = extend<17>(value)
low  = truncate<8>(wide)

```

`extend<N>` preserves signedness behavior and `truncate<N>` keeps low bits. Neither changes the type family.

At an explicitly typed output, local, register write, storage write, child binding, struct field, or declared function-return boundary, the width may be obtained from that boundary while the conversion remains explicit:

```

low : u8 = truncate(wide)
wide_again : u17 = extend(low)

```

The contextual and explicit-width forms normalize to the same typed resize. An inferred local, a nested arithmetic operand, or a mismatched type family has no such context and must keep `truncate<N>` or `extend<N>`.

4.6 Fixed point

Concrete spellings expose integer and fractional widths directly:

Spelling	Canonical type	Narrowing policy
SF8.8	<code>fixed<16,8></code>	wrap
SF_Sat8.8	<code>fixed_sat<16,8></code>	saturate
UF8.8	<code>ufixed<16,8></code>	wrap
UF_Sat8.8	<code>ufixed_sat<16,8></code>	saturate

For signed values the integer part includes the sign bit. SF8.8 therefore represents -128.0 through 127.99609375 in steps of 2^{-8} .

Arithmetic is widened before any storage conversion. Fixed addition and subtraction require equal fractional widths; multiplication returns total width $W1+W2$ and fractional width $F1+F2$. Target overflow policy applies only at an explicit or contextual storage conversion, not after every intermediate node.

Direct fixed-point literals must be exactly representable and in range, even for a saturating target. Loss is explicit:

```
y = quantize<SF_Sat8.8>(value) {
    round nearest_even
    overflow saturate
}
```

The rounding mode is mandatory; there is no implicit rounding default.

round mode	Discarded fractional part
<code>toward_zero</code>	Truncate toward zero.
<code>floor</code>	Round toward negative infinity.
<code>away_zero</code>	Round away from zero if any fractional part remains.
<code>nearest_even</code>	Round to nearest; on an exact tie choose the even stored integer.

Form	Target and overflow behavior
<code>quantize(value, MODE)</code>	Target comes from an unambiguous fixed-point assignment/return context; overflow defaults to that target type's policy.
<code>quantize(value) { round MODE overflow POLICY }</code>	Contextual target; both clauses are required.
<code>quantize<T>(value) { round MODE overflow POLICY }</code>	Explicit fixed-point target; both clauses are required.

POLICY is `wrap` (modular truncation to the target bit width) or `saturate` (clamp to the target's minimum/maximum). Ordinary `fixed`/`ufixed` targets default to `wrap`; `fixed_sat`/`ufixed_sat` targets default to `saturate` only in the short two-argument form. The block's explicit policy takes precedence. Conversion order is exact rescale, rounding, then overflow. `fixed_raw(0x0C22)` creates a target-context raw pattern; `fixed_to_raw` exposes the stored integer.

An exact fixed-point `dot(a,b)` retains the full accumulator. If the destination reduces fractional precision, give one post-accumulation rounding mode:

```
out y : SF_Sat8.8
y = dot(coefficients, samples, floor)
```

4.7 Vectors and structs

`vec<N,T>` has a compile-time length:

```
in samples : vec<4,u8>
```

An explicitly typed boundary can use a non-empty vector literal or contextual replication:

```
pair      : vec<2,u8> = [left, right]
zero_values : vec<4,u8> = repeat(0)
zero_values_2 : vec<4,u8> = repeat<4>(0)
```

Every element must have the exact contextual element type. `repeat(value)` requires the destination vector to provide its length; `repeat<N>(value)` makes the length explicit. Both forms have the same vector sequence order and statically generated hardware.

Constant indexes are checked during elaboration. Runtime reads are accepted only when conservative range analysis proves every encoded value in range; ZLang does not silently mask or clamp. A rule may update one range-proven element of a one-dimensional reg `vec<N,T>` atomically. Nested paths, multiple dynamic writes to the same register action group, and runtime-selected instances remain unsupported; see [Sequential logic, rules, and storage](#).

Structs are nominal products:

```
struct Packet {
  data : u64
  last : bit
}

packet = Packet { data = payload last }
```

Constructors are complete and may use same-name field punning. An immutable update rebuilds the same nominal struct and preserves fields not named in the update:

```
completed = packet with { last = 1 }
```

Unknown or duplicate fields and values of the wrong exact field type are rejected. This is a pure value expression, not mutation. Recursive layouts are still rejected. Generic structs are covered in [Expressions, functions, and generics](#).

Structural `==` and `!=` are available for exact matching struct and vector types. Comparison is recursive over every field/element and returns `bit`; it does not perform conversion, resizing, or nominal coercion.

An immutable nominal struct can also be deconstructed exhaustively:

```
Packet { data, last } = packet
```

Every declared field must appear exactly once, under its declared name, and the new immutable bindings may not shadow another symbol. Deconstructing produces ordinary typed field values; it does not add tuple patterns, partial patterns, renaming, mutation, or runtime matching.

4.8 Slicing, concatenation, and representation

An inclusive, compile-time bit slice returns raw bits:

```
in word : u16
out byte : bits<8>
byte = word[15:8]
```

Both `MSB` and `LSB` are compile-time integer expressions. The result is `bits<MSB-LSB+1>`. Reversed, negative, runtime, or out-of-range bounds are errors; slicing never resizes, reinterprets, or clamps its operand.

A `bits<N>` value also supports concise single-bit selection:

```
in raw : bits<24>
out lsb : bit = raw[0]
out msb : bit = raw[23]
```

Packed indices use conventional bit numbering: index zero is the least-significant bit, exactly as `[0:0]`. A runtime unsigned selector is legal only when its compiler-proven interval lies entirely within the packed width; there is no implicit modulo, clamp, or out-of-range hardware behavior. A fixed-width runtime window uses LSB-based `+` spelling:

```

in raw    : bits<16>
in offset : u3
out byte  : bits<8>
byte = raw[offset +: 8]

```

WIDTH is a positive compile-time integer. The compiler requires `offset + WIDTH <= packed_width` for every possible offset. Vector element zero likewise occupies the least-significant packed element region.

`concat(a,b,...)` accepts at least two operands. When every operand is a vector with the exact same element type it returns one longer vector, preserving source and element order:

```

in left  : vec<2,u8>
in right : vec<3,u8>
out all  : vec<5,u8>
all = concat(left, right) // left[0], left[1], right[0], right[1], right[2]

```

Otherwise no operand may be a vector and all operands must be recursively bit-packable. The result is `raw bits<N>` and the first operand occupies the most-significant bits:

```

joined = concat(high, low) // high is the MSB portion

```

Each scalar operand is typed independently, before the result width is known. For example, `concat(0,1)` is `bits<2>` and `concat(3,0)` is `bits<3>`. Neither an assignment target nor a function caller may widen an operand or the completed concatenation implicitly.

Exact packed fill constants are available without manufacturing a numeric literal of the desired width:

```

clear_mask : bits<24> = zeros<24>
set_mask   : bits<24> = ones<24>

```

The width is a positive compile-time integer expression. Both forms produce exact `bits<N>` constant values; they are not vector replication. The separate `zero<x>/ones<x>` spellings inside an `equiv` declaration remain typed pattern constants rather than hardware expressions.

Vector/scalar mixtures and different vector element types are errors. Convert a vector explicitly with `bitcast<bits<W>>` when raw bit assembly is intended.

`reshape(value)` obtains a vector target shape from an explicitly typed context; `reshape<T>(value)` supplies it directly. Reshape recursively enumerates leaves outer-to-inner, then rebuilds the requested shape without packing them:

```

in matrix : vec<2,vec<4,u8>>
out flat   : vec<8,u8>
flat = reshape(matrix)
matrix_again = reshape<vec<2,vec<4,u8>>>(flat)

```

Source and target must be vectors with the same leaf count and exact same leaf type. Reshape has no endianness and performs no numerical conversion.

`bitcast<T>(value)` exposes or reconstructs the exact stored representation when source and target have equal packed width and both are recursively bit-packable and non-enum. It never resizes, sign-extends, truncates, rescales, rounds, or saturates:

```

raw    : bits<8> = bitcast<bits<8>>(signed_value)
signed : s8      = bitcast<s8>(raw)

```

`pack(value)` and `unpack<T>(raw)` remain supported low-level compatibility spellings and normalize to the same typed Bitcast operation. Signed integers and signed fixed-point values use their stored two's-complement pattern.

Aggregate layout is deterministic and backend-independent:

- struct field zero (declaration order) is at the MSB;
- vector element zero is at the LSB;
- tuple component `item0` is at the LSB;

- nested vectors and tuples apply the LSB-first rule recursively;
- nested structs retain declaration-order/MSB-first field layout.

The initialized `rom<T,N>` image format uses this same layout for each word. Images contain one exact-width binary word per line with address zero first; they do not inherit host byte order. Consequently simulator values and Direct-SV `$readmemb` agree on nested aggregate and fixed-point ROM contents. See [Initialized synchronous ROMs](#).

Scalar numeric/raw types and recursively bit-packable structs, vectors, and structural tuples are accepted. Nominal enums, including an aggregate containing an enum, are intentionally rejected by `bitcast`, `pack`, and `unpack`. Enum ordinal storage remains available through ordinary typed values and `reshape`, but raw construction could create an invalid ordinal; no validity policy for that conversion is defined.

At explicitly typed assignment/storage boundaries, ZLang may insert an equal-width raw bitcast only when either source or target is `bits<N>` or `flat vec<N,bit>`. This makes raw wiring concise while preserving the rule that direct signed-to-unsigned, integer-to-fixed, resizing, and rescaling conversions are never implicit. Raw conversion does not participate in function arguments, operator overloads, generic inference, switch branch unification, or protocol compatibility.

Scalar `reduce(&, value)`, `reduce(|, value)`, and `reduce(^, value)` reduce the stored bits of `bit`, `bits<N>`, `uN`, or `sN` and return `bit`. `parity(value)` is the concise XOR form and also accepts `vec<N,bit>`. Vector reduction still reduces elements and returns the element type; fixed-point and struct scalars are not representation-bit reduction operands.

4.9 Boundaries

Unproven or out-of-range runtime packed selection, runtime `[MSB:LSB]` bounds, reversed/out-of-range slicing, zero or unresolved packed widths, one-operand or heterogeneous-vector concatenation, runtime reshape, unequal-width bitcast, enum bitcast, and external top enum inputs remain deliberately rejected. These boundaries are diagnosed rather than inferred by a backend. See the [syntax support matrix](#).

5 Expressions, functions, and generics

ZLang expressions are pure typed hardware values unless they explicitly contain a timing construct. Assignment requires an exact canonical target type.

5.1 Operators and selection

From highest to lowest precedence:

1. calls, parentheses, aggregate selection, conversions, functional forms;
2. unary `-` and logical `!`;
3. `*`, and compile-time `/`;
4. `+`, binary `-`;
5. `<<`, `>>`;
6. `&`, then `^`, then `|`;
7. `==`, `!=`, `<`, `<=`, `>`, `>=`;
8. `&&`, then `||` in compile-time/equivalence guards;
9. right-associative `condition ? true_value : false_value`.

Unary `-` is implemented for numeric scalar/fixed types and may be overloaded for a nominal struct. It is not a general implicit-cast mechanism. Runtime logical `!` accepts exactly `bit` and returns `bit`; it is equivalent to an exact comparison with zero. `/`, `&&`, and `||` are available only where the compile-time/equivalence sublanguage accepts them; they do not infer runtime divider or general truthiness for multi-bit hardware values.

Two-way runtime selection can use `?:` or `mux`:

```
y = select ? a : b
y = mux(select, a, b)
```

The condition is `bit` and both arms have one exact type. Numeric `switch` requires unique fitting keys and an `else` arm:

```
y = switch opcode {
```

```

0 => a
1 => b
else => 0
}

```

A switch over a nominal enum uses qualified members and must cover every member. It needs no catch-all arm, which keeps a newly added state from silently taking an old default path:

```

next = switch state {
  State.Idle => State.Header
  State.Header => State.Payload
  State.Payload => State.Done
  State.Done => State.Idle
}

```

Numeric keys in an enum switch, members from another enum, duplicate members, and incomplete coverage are errors. Numeric switches require an `else` arm.

Nominal tagged unions use the parallel exhaustive `match` expression. Each arm names its union and variant and binds the exact source-declared payload names; there is no default arm or hidden priority:

```

y = match message {
  Message.Idle => 0
  Message.Data { value } => value
}

```

This has the same semantics as an exact-width `switch` over the union tag and typed field values.

Exact matching structs, vectors, and structural tuples also support `==` and `!=`. These comparisons recursively combine the ordinary exact leaf comparisons and return `bit`; mismatched nominal structs, vector lengths, tuple component order/arity, or element types are errors.

5.2 Aggregate value ergonomics

Vector literals and replication use their explicit destination type:

```

pair          : vec<2,u8> = [left, right]
zero_values   : vec<4,u8> = repeat(0)

```

`repeat<N>(value)` may state the compile-time length explicitly. The contextual and explicit forms have the same statically generated vector semantics. Struct update is likewise a pure expression over a nominal value:

```

next = current with { valid = 1 payload = replacement }

```

It preserves omitted fields and produces a complete typed struct value; it does not create mutable records or a backend-specific update operation. Exhaustive `Struct { field, ... } = value` destructuring similarly produces ordinary immutable field bindings. It requires every field exactly once and does not provide partial patterns, renaming, or runtime pattern matching. Flat tuple destructuring `(first, second) = pair` likewise evaluates one RHS and introduces fresh immutable projections. Nested, wildcard, and rest patterns remain outside this bounded surface.

5.3 Pure functions

Functions are top-level, pure, and combinational. The smallest body is one result expression:

```

fn tap(sample : u8, coefficient : u8) -> u16 {
  sample * coefficient
}

```

They may call other non-recursive pure functions. They cannot capture module ports or contain state, clocks, delays, protocols, or pipelines. Return types may be explicit or inferred for both ordinary and generic functions:

```

fn make_tag(high : bits<2>) {

```

```

    concat(high, zeros<2>, ones<2>)
}

```

An inferred body is checked without a caller-provided expected type, and its exact final-expression type becomes the function signature. Forward references are resolved independent of declaration order. Direct or indirect inferred return cycles are rejected; ZLang does not attempt a recursive type fixed point. For ordinary functions, explicit and inferred spellings with the same exact signature identify the same callable. Generic specialization identity continues to include the generic declaration form and its exact specialization arguments.

A body may introduce sequential immutable bindings without `let`, `const`, or `return`; the final expression is the result:

```

struct Pair<type T> { left : T right : T }

fn duplicate_sum<type T>(a : T, b : T) {
    exact = a + b
    Pair { left = exact right = exact }
}

```

Each binding has its exact inferred type, may reference parameters and earlier bindings, and cannot shadow or be reassigned. Forward references and module captures are rejected. This syntax has the same typed behavior as the nested expression spelling.

Source units are declaration-order independent: functions, structs, enums, and modules may be declared after their users, and a shipped library unit may contain declarations without a hardware top. Parameterized modules can state compile-time obligations at specialization:

```

module Queue<type T,D=4>
where D >= 2 && is_power_of_two(D) {
    // ...
}

```

The constraint is checked against exact type/value arguments and creates no runtime hardware.

5.3.1 Typed constants and statically selected functions

Module and generic-function specializations may carry required named compile-time constants and pure function references:

```

fn widen(x : u8) -> u9 { extend<9>(x) }

fn transform_with<
    type A,
    type B,
    operation : fn(A) -> B
>(x : A) {
    operation(x)
}

module ImageBank<type T,N,IW,image : vec<N,T>> {
    clock clk
    reset rst
    in address : uint<IW>
    out data : T

    rom table : rom<T,N> { read_latency 1 init image }
    table.read_address = address
    data = table.read_data
}

```

These argument kinds are always named: `image=image` and `operation=fn widen`. A generic target may be selected explicitly, for example `operation=fn widen_as<A=u8,B=u9>`. Constants must have the exact declared, recursively

bit-packable non-enum type and must evaluate completely during elaboration. A function reference must be pure and match the exact parameter and return types; it is not a runtime function pointer and cannot be stored, returned, compared, or carried through hardware ports.

Type and integer shape parameters resolve before constant/function parameters. Their identity uses canonical constant values or exact callable identities and the locked dependency content, not source spelling. Defaults for constant/function parameters are unsupported.

5.4 Compile-time functions and selection

Compile-time evaluation is bounded and deterministic. These structural intrinsics are built into the language rather than supplied by stdlib overloads:

- `length(vector)`;
- `floor_log2(n)`, `ceil_log2(n)`, `index_width(n)`, `is_power_of_two(n)`;
- `pi()`, `sin(x)`, `cos(x)`, `log2(x)`, `log(base,x)`, `exp(x)`, and `sqrt(x)`.

Real intrinsics use a versioned decimal evaluator rather than host binary floating point. A non-integral real enters hardware only through explicit fixed-point quantization. `1 / sqrt(x)` is the compile-time reciprocal square root form used when generating seed or approximation tables.

`index_width(n)` is the storage width for an index into `n` elements: `max(1, ceil_log2(n))`. Its argument must be a positive compile-time integer expression. Integer parameter defaults are resolved in declaration order and may reference only earlier parameters:

```
module Table<N=8,IW=index_width(N)> {
    // Ports and implementation omitted.
}
```

A forward reference such as `module Bad<IW=index_width(N),N=8>` is rejected.

Specialization-time `if` accepts value parameters and canonical type parameters:

```
fn type_flag<type T>(x : T) {
    if T == u8 { 1 } else { 0 }
}

module Stage<D=1> {
    out y : u8
    if D == 1 { y = 1 } else { y = 2 }
}
```

Only the selected branch is semantically checked. Runtime signals are rejected in compile-time conditions with a suggestion to use `?:`, `mux`, `switch`, or a guarded action.

5.5 Functional datapath

Ranges are half-open and compile-time bounded:

```
products = generate(i in 0..8) a[i] * b[i]
doubled  = map(i in 0..8) { a[i] + a[i] }
y        = reduce(+, products)
z        = sum(i in 0..8) a[i] * b[i]
d        = dot(a, b)
```

`generate` and `map` produce `vec<N,T>`. `reduce` supports `+`, `*`, `&`, `|`, and `^`; `sum` is addition reduction. Reduction order is a deterministic source-order balanced tree, so finite-width intermediate types are stable across backends. `dot` uses full-width products followed by that same reduction.

For a nominal struct element, only `sum/reduce(+, ...)` is defined. Every node uses the ordinary exact operator `+` overload for the two types produced by its children; that result type becomes the input type at the next level. Reduction order is deterministic and preserves the exact balanced topology and intermediate types described here, regardless of how the compiler represents a large bounded reduction.

There is no component-wise field rule, implicit conversion, identity insertion, or special case for `Complex`. The balanced tree is not reassociated, and explicit quantization remains exactly where the source or overload body placed it.

Range bounds may use resolved module value parameters. Runtime ranges, empty value-producing ranges, and unbounded generation are rejected.

Compile-time iterator arithmetic is mathematical integer arithmetic. It does not require hardware `extend` / `truncate` noise:

```
reversed = generate(i in 0..48) bits[47 - i]
```

For a vector, `values[first..past_last]` is a half-open compile-time range and has the same ordered elements as the corresponding vector literal. It is not a runtime slice and does not change packed bit-slice syntax `raw[MSB:LSB]`.

A functional iterator is a compile-time integer and may feed a generic value specialization directly or through a bounded constant expression:

```
fn coefficient<K>() -> u8 { K }

values = generate(k in 0..4) coefficient<K=k>()
next    = map(k in 0..4) { coefficient<K=k+1>() }
total   = sum(k in 0..4) coefficient<K=k>()
```

The evaluated value, not the binder spelling, participates in specialization identity. Runtime values are not specialization constants, and an iterator binder cannot shadow a module parameter.

5.5.1 Scalable bounded functional elaboration

Large compile-time `generate`/`map` regions with a binder-invariant result type are supported within the same bounded generation budget (currently at most 4096 logical elements). Every element is checked with the ordinary exact type rules; generation does not become a runtime loop or defer correctness to an RTL tool. Callable specializations remain ordinary monomorphic definitions, and each use retains its own source origin.

A whole-vector IFFT64 can express an outer 64-lane `generate`, 64 exact products per lane, nominal complex reduction, and one final quantization. This describes a large combinational hardware structure; it does not imply a production streaming architecture or a particular physical implementation.

A fully compile-time generated vector may initialize an immutable ROM without external source generation:

```
fn coefficients<N>() {
    generate(i in 0..N) truncate<8>(i * i)
}

rom table : rom<u8,N> {
    read_latency 1
    init coefficients<N=N>()
}
```

The complete transitive function/intrinsic dependency identity participates in the ROM and companion hashes. Compile-time functions have no filesystem, network, process, clock, state, random, or environment access. The resulting immutable image must have exactly the declared vector shape and element type.

5.6 Typed bit and collection composition

Bit slicing and composition are expressions rather than backend spellings:

```
high   : bits<4> = word[7:4]
flag   : bit    = word[0]
joined : bits<8> = concat(high, word[3:0])
same   : u8     = bitcast<u8>(joined)
flat   : vec<8,u8> = reshape(matrix)
```

Slices use inclusive compile-time [MSB:LSB] bounds. A single packed-bit selection `raw[index]` returns `bit`, uses conventional LSB-zero numbering, and allows a range-proven runtime unsigned index. A fixed-width packed slice `raw[offset +: WIDTH]` returns `bits<WIDTH>` after the same bounds proof. Vector sequence indexing uses the same low-first convention: `vec` element zero occupies the least-significant packed element region. Scalar `concat` places its first operand at the MSB; homogeneous-vector `concat` preserves collection order and returns a vector. `reshape` changes only nested vector shape. `bitcast<T>` uses the canonical layout described in [Types and numerics](#): struct declaration field zero is at the MSB, while vector element zero and tuple `item0` are at the LSB. It requires exact packed width and performs no resize or scale change. A source or target containing a nominal enum is rejected; enum representation is not a public bitcast contract. `pack` and `unpack<T>` remain low-level compatibility spellings.

For representation-bit checks, `reduce(^, scalar)` and `parity(scalar)` operate on the stored bits of `bit`, raw bits, and signed/unsigned integers. Reducing a general vector retains the element-wise reduction semantics.

5.7 Generic structs, functions, and operators

Generic parameters have explicit kinds:

```
struct Pair<type T> { left:T right:T }

fn duplicate<type T>(x:T) { x + x }

module Use {
  in x : u8
  out y : u9
  y = duplicate(x)
}
```

Calls support exact call-site inference or explicit specialization. Generic struct constructors infer their specialization from fields where unambiguous. Every accepted call is monomorphized; there is no runtime dispatch.

A direct integer literal may use a concrete formal argument type after explicit generic arguments or non-literal arguments have fixed that type:

```
first = same<T=u16>(1)
second = combine(1, existing_u16)
third = combine(existing_u16, 1)
```

If a type is inferable only from literals, each literal first receives its own minimum-width exact type and normal exact unification applies. Argument order does not select a wider type. Compound expressions never receive this literal-only contextual treatment.

Libraries may overload binary `+`, `-`, `*` and unary `-` for nominal structs. Built-in scalar/fixed operations cannot be replaced. Resolution uses exact type unification, prefers an exact concrete overload over a generic one, performs no implicit conversion, and enforces nominal-owner coherence. Each specialization is deterministic, and Direct-SV emits one function automatic per specialization rather than cloning the body at every call site. Nominal reductions preserve their overload-defined operation and are not reassociated as scalar arithmetic.

`std.math.complex` is ordinary ZLang source defining `Complex<T>`, arithmetic operators, `Butterfly<S,D>`, and explicit component quantization. For example:

```
Complex<fixed<18,16>> * Complex<fixed<16,14>>
-> Complex<fixed<35,30>>
```

The scale-30 result cannot be added to a scale-16 value until explicitly quantized. Its `complex_sum(values)` helper is a discoverability alias that returns the same typed `Reduce` as `sum(values)`; it adds no compiler behavior. Generics and nominal reductions do not imply rescaling, rounding, saturation, or casts.

Traits, runtime polymorphism, generic methods, function-name overloading, higher-order functions, recursion extensions, and implicit conversions are unsupported.

6 Sequential state, rules, pipelines, and storage

ZLang keeps combinational values, next-state actions, timing, storage, and physical clock ownership explicit in the typed design. Synchronous active-high reset is the default. Each independently declared clock has one reset contract and may own ordinary state in the same module.

6.1 Clock and reset

Single-domain modules declare one pair:

```
clock clk
reset rst
```

Because newlines are ordinary whitespace, the compact adjacent spelling is equivalent and introduces no combined declaration or inferred domain:

```
clock clk reset rst
```

Clock and reset are declared together. Multiple domains require explicit reset, port, and state ownership wherever inference would be ambiguous:

```
clock source_clock
reset source_reset @source_clock

clock destination_clock
reset destination_reset @destination_clock

reg source_count : u32 @source_clock = 0
reg destination_count : u32 @destination_clock = 0

rule TickSource @source_clock when 1 {
    source_count <- truncate<32>(source_count + 1)
}

rule TickDestination @destination_clock when 1 {
    destination_count <- truncate<32>(destination_count + 2)
}
```

An unannotated stateful declaration remains concise in a single-clock module. In a multi-clock module its domain may be inferred from one unique destination or dynamic operand domain; otherwise analysis reports ZL-DOMAIN-AMBIGUOUS. Constants are domain-neutral. Combining dynamic values from different domains, including through an unannotated local, reports ZL-DOMAIN-CROSSING.

Rules and priorities are scheduled within a domain. Rules in unrelated domains do not acquire cross-clock mutual exclusion or atomicity. FSM-generated state and rules inherit the FSM annotation. Exact pipeline(N) @clk { ... }, FIFO, memory, ROM, and CSR state similarly retain one resolved owner; inserted pipeline and balancing registers never act as CDC.

Implicit clock-domain crossing is always rejected. See [Hierarchy and protocols](#).

The recommended asynchronous spelling asserts immediately and releases only after two active clock edges:

```
clock clk
async reset arst_n @clk { polarity active_low }
```

Normal state transition resumes on the third edge after external deassertion. Use the externally synchronized release contract only when the external source already guarantees synchronous deassertion to this exact clock. It removes the internal 2FF while retaining asynchronous reset for state with a declared reset value. The low-level reset ... { mode asynchronous ... } spelling retains raw native deassertion for compatibility. Exact polarity, falling-edge behavior, hierarchical propagation, and unsupported formal/target routes are documented in the [physical clock/reset contract](#).

6.2 Registers and next state

```
reg count : u8 = 0
count <- truncate<8>(count + 1)
```

The explicit multi-clock spelling places the annotation before the initializer:

```
reg count : u8 @datapath_clk = 0
```

Expressions read committed beginning-of-cycle state. All accepted <- updates become visible together at the active edge. A register without an accepted update holds. Reset restores the declared constant. Multiple unordered writers are rejected.

The reset value is optional. `reg retained : u8` has no generated reset branch or reset sensitivity; its hardware value is unspecified until written. Native two-state simulation starts it from a deterministic zero seed for tool parity, which is not a power-up or reset guarantee in generated RTL.

Control registers may use nominal enums. Reset and next-state values must be members of the exact same declaration, and an exhaustive enum switch is the preferred finite-state-machine spelling:

```
enum Phase { Idle Active Done }
reg phase : Phase = Phase.Idle
phase <- switch phase {
  Phase.Idle => start ? Phase.Active : Phase.Idle
  Phase.Active => finish ? Phase.Done : Phase.Active
  Phase.Done => Phase.Idle
}
```

Native simulation and Direct-SV share the declaration-order ordinal encoding.

For control-oriented state, fsm is concise syntax for the same enum register and guarded atomic rules:

```
enum TxPhase { Idle Header Payload Done }

fsm phase = TxPhase.Idle {
  Idle {
    when start -> Header { remaining <- length }
  }
  Header { when header_sent -> Payload {} }
  Payload {
    priority {
      when abort -> Idle {}
      when last -> Done {}
    }
  }
  Done { -> Idle {} }
}
```

The qualified initial member in `fsm phase = TxPhase.Idle` infers the exact nominal enum type. The explicit compatibility spelling `fsm phase : TxPhase = Idle` remains accepted; neither form infers from guards, actions, or transition targets.

Every member appears exactly once. A state body is hold, one transition, or an explicit priority block; source order never creates hidden priority. The state update and transition actions form one ordinary atomic rule, reset restores the declared initial member, and direct writes to the generated state register are rejected. fsm uses the ordinary scheduler and does not add a second scheduler or runtime procedural if.

The Wi-Fi IeeeIFFTInputStrip is a second source example: its Data/Flush FSM atomically starts and retires a 64-transfer zero-frame flush, holds under backpressure, and returns to Data after the last accepted sample. Mid-flush reset restores the declared initial state and discards the incomplete protocol epoch.

6.2.1 Runtime-indexed vector register updates

A one-dimensional vector register supports one range-proven runtime element write in an action group:

```

reg samples : vec<64,u8> = generate(i in 0..64) 0

when capture {
  samples[index] <- value
}

```

The update replaces one selected element of the beginning-of-cycle vector value and commits atomically with the containing action. The unsigned index must be statically proven in range. A second dynamic or static write to the same vector register conflicts; nested paths, slices, runtime-selected instances, and automatic memory inference remain outside this bounded form.

A rule guard may establish the required bound through a simple unsigned comparison and conjunction:

```

when (index < 64) & capture {
  samples[index] <- value
}

```

That fact is scoped to this rule only. The bounded refinement understands these simple comparisons/conjunctions; `index <= 64`, disjunctions, and uses outside the guarded rule do not prove the required `0..63` range.

6.3 Delays and fixed pipelines

```

delayed = delay<2>(value)
piped = pipeline(3) { expression }

```

`delay<N>` adds exactly `N` zero-reset scalar stages. `pipeline(N)` is an exact visible-latency contract: for a supported pure scalar expression DAG, planning may place real computation boundaries within those `N` cycles and inserts the required reconvergent-path balancing. It never changes the requested latency or II, and `pipeline(1)` remains exactly one cycle. Unsupported nodes fail closed; the emitter does not fall back to an unreported whole-expression output delay. Outside a scheduled pipeline region, non-timeless operands at an operator must still be latency-aligned explicitly.

Compiler-selected latency/resource alternatives use `implement { expression intent { ... } }`. Scalar pipeline (auto) is retired; protocol transform `pipeline(auto, ...)` is a separate ready/valid construct.

6.4 Exact module timing contracts

A scalar datapath module may publish one exact contract shared by all of its wire outputs:

```

module TimedChild {
  clock clk
  reset rst
  in x : u8
  out y : u8

  timing {
    latency 4
    ii 1
  }

  y = pipeline(4) { x }
}

```

The contract describes observable behavior; it does not itself insert registers or authorize retiming. Constants are timeless, ordinary input-dependent logic is known at latency zero, and explicit `delay<N>/pipeline(N)` adds exactly `N`. A supported pipeline region may place and balance required register cuts automatically. Timeless values may join a known path; joins outside that graph still require equal known latency.

Registers, rules, FIFO/memory/ROM observations, protocols, and uncontracted child outputs are unknown under the current bounded contract. A positive latency requires exactly one clock/reset domain. Only `ii 1` is accepted. A contracted child adds its declared latency to the common known latency of its bound scalar inputs; an unaligned parent join is rejected instead of silently balanced.

This is distinct from `implement` constraints, target estimates, and measured evidence. Implementation selection and profiles must preserve the exact module contract as immutable public behavior.

6.5 Guarded atomic actions

The concise action form is:

```
when increment {
  count <- truncate<8>(count + 1)
}
```

Named compatibility spelling remains supported:

```
rule increment_count when increment {
  count <- truncate<8>(count + 1)
}
```

Action blocks may select effects recursively with runtime `when`, `else when`, and `else`:

```
fault_update: when fault {
  when valid_state {
    overflow_state <- 1
  } else {
    valid_state <- 1
    overflow_state <- 0
    sticky_state <- fault_code
  }
} else when clear_valid {
  valid_state <- 0
  overflow_state <- 0
} else when clear_overflow {
  overflow_state <- 0
}
```

An `else` binds to the nearest unmatched `when`, and an `else when` chain selects its first true predicate. A false `when` without an `else` contributes no effects; later statements in the enclosing action block are still considered, so multiple sibling conditionals can contribute to the same transaction.

Read this as two operations that never feed back into each other: predicates first choose the effects, then the scheduler either commits the complete chosen transaction or commits nothing. Storage readiness and rule conflicts belong only to the second operation; they can suppress a transaction but can never make an `else` branch run.

Every guard and operand reads the same pre-edge snapshot. The complete tree defines one atomic action: selected effects and unconditional surrounding effects commit together or not at all. If no effect is active, the action does not fire. Source order creates neither state visibility nor implicit priority, and reset suppresses the complete action.

Branch choice depends only on the predicates. If the selected branch contains an illegal FIFO or memory action, the whole action group is suppressed; it does not fall back to an `else` branch because another branch's storage action would be ready. The scheduler checks conflicts and storage legality using only active effects. Scalar output writes are scheduling resources too: a rule that loses a priority conflict on an output cannot commit its otherwise unrelated state effects.

Each runtime action condition must have exact type `bit`. Opposite structured branches may write the same resource, but writes on potentially overlapping paths are rejected; the compiler does not attempt arbitrary Boolean theorem proving. An explicitly empty branch is a valid no-op, while an entire action tree with no possible effect is rejected.

Runtime action selection is distinct from both other selection forms:

- compile-time `if` chooses declarations or values during elaboration and is not an action statement;
- `condition ? a : b`, `mux`, and `switch` select pure combinational values;
- runtime `when` selects effects that participate in one cycle-level atomic transaction.

Rules read beginning-of-cycle state. If the guard is true, all resource actions are legal, and scheduling permits the rule, its actions commit atomically. Reset suppresses all rules. Conflicting writers require explicit priority:

```
priority {
  clear: when clear_request { count <- 0 }
  increment: when increment_request {
    count <- truncate<8>(count + 1)
  }
}
```

Priority suppresses a lower enabled rule only when it conflicts with a fired higher rule. It is not procedural if/else: nonconflicting enabled rules may fire together.

A simple ordered chain may instead be written as:

```
priority first > second > third
```

This syntax expands to the adjacent edges `first > second` and `second > third`. It does not invent extra edges or infer priority from source order; retain the block/pair form for any other priority graph.

6.6 FIFOs

```
fifo queue : fifo<u8,4>
queue.data = rx.payload
queue.push = rx.transfer
queue.pop = tx.transfer
```

Read-only observations include `.front`, `.count`, `.full`, `.empty`, `.ready`, `.valid`, `.overflow`, and `.underflow`. A legal simultaneous pop/push while full preserves occupancy and ordering. An empty pop cannot consume a same-cycle push.

FIFO depth may be a positive compile-time value expression. Backends receive the resolved integer, not a hardware parameter.

Rules can instead own FIFO operations atomically:

```
rule replace when rotate {
  queue.pop()
  queue.push(queue.front)
  accepted <- 1
}
```

Do not mix rule-owned actions with globally driven `.push/.pop` controls. An illegal resource action suppresses the whole rule.

Rule scheduling selects the lexicographically highest-priority **legal atomic set**, not each rule independently. Thus a push on a full FIFO may fire with a compatible lower-priority pop on the same edge; a push alone remains blocked. Scheduling remains exact within a bounded compile-time work budget. If that budget is exceeded, compilation reports ZL-STATE-SCHEDULER-LIMIT rather than emitting an approximate circuit.

6.7 Writable memories

One-read/one-write memories may optionally expose a byte write mask. The one-cycle, reset-cleared form is the default:

```
in write_mask : bits<4>
memory table : mem<u32,256> {
  read_latency 1
  collision write_first
}
table.read_address = read_address
table.write_enable = write_enable
table.write_address = write_address
```

```
table.write_data = write_data
table.write_mask = write_mask
```

The mask type is exactly $\text{bits}\langle\text{ceil}(\text{element_width} / 8)\rangle$. `write_mask[0]` controls the least-significant byte. For a width that is not a multiple of eight, the final mask bit controls only the remaining live most-significant bits; padding bits are never stored. Disabled lanes retain their old bits. For a same-address `write_first` access, the registered read result is the post-mask merged word, not the unmasked input. Omitting `write_mask` preserves the full-word write behavior. Consequently, unmasked and masked memory elements may have any positive recursively bit-packable, non-enum width.

Rule-owned memories accept the same operation as an optional third operand:

```
rule store when enable { table.write(address, data, byte_mask) }
```

Within a masked rule-owned memory, the compatible two-operand form `table.write(address, data)` means an all-lanes write. The mask does not add a read enable, another port, a new clock domain, or target-specific BRAM mapping.

`read_latency` counts active edges in the memory's read domain between a captured address and the visible result. These are compiler/simulator/Direct-SV contracts, not promises that a particular FPGA RAM primitive will be inferred:

Storage form	Accepted <code>read_latency</code>	Behavior and physical boundary
Globally controlled mem, including named same-clock ports	0..16	0 is combinational; 1 is registered; 2..16 add exact read-domain register stages.
Rule-owned mem	Exactly 1	A selected read action captures at an edge and holds its result otherwise.
<code>async_mem 1W1R</code>	1..16	Reader-domain registered output, including unrelated clocks. Generic Direct-SV supports <code>old</code> ; stronger cross-clock collision claims need exact target evidence.
Immutable rom	Exactly 1	Synchronous table read.
Internal memory of <code>async_fifo</code>	Fixed 1	The FIFO prefetch controller accounts for the registered read; this is not a user-settable FIFO latency.

The explicit Xilinx 7-Series synchronous-memory physical route accepts only `read_latency 1` and matching reset, mask, port, and collision capability. Other legal latencies can use generic RTL under a preferred policy; a required native route rejects them. `read_latency 0` never claims synchronous BRAM.

6.7.1 Named same-clock ports

Leaving the body without port declarations uses the implicit 1R1W controls and their generated behavior. A memory may instead declare up to eight named logical ports:

```
memory table : mem<u32,1024> @clk {
    read_write_port a
    read_write_port b
    init RESET_WORD
    read_latency 1
    collision old
    write_priority a > b
}

table.a.read_enable = a_read
table.a.write_enable = a_write
table.a.address = a_address
table.a.write_data = a_data
a_data_out = table.a.read_data
```

`read_port`, `write_port`, and `read_write_port` state the exact access shape. All named ports of an ordinary `mem` have one resolved clock domain. When more than one port can write, `write_priority` must list every writer exactly once. Priority affects only writes to the same address; simultaneous writes to different addresses both commit. `collision old`, `new`, and `no_change` select the same-clock registered-read result. The older spellings `read_first` and `write_first` remain accepted aliases for `old` and `new`.

Named-port masks retain the arbitrary-bitwidth lane rule above. Mixed port widths and asymmetric depths are rejected. Rule-local memory actions remain a property of the implicit 1R1W form and cannot be mixed with named ports.

An optional `init VALUE` supplies one exact compile-time element value for every cell. It may be a typed module value parameter. With `contents clear`, generic/register-array hardware writes that value to every cell on reset. With `contents preserve`, it is only the deterministic power-up value; FPGA block RAM maps it to bitstream initialization and does not pretend that a runtime reset rewrites the array. Omitting `init` retains the exact zero value used by the declared reset/initialization policies. Full per-address images are the role of immutable rom in the language contract.

When an exact target memory resource matches the requested shape, it may be selected. Otherwise, a 1W+nR shape may use coherent replicated 1R1W storage. Other same-clock multiwrite shapes may use a deterministic register array, read muxes, and priority gates when the logical storage is at most 4096 bits. Larger unsupported shapes fail with a diagnostic recommending explicit banking, arbitration, or standard-library composition. No automatic banking is performed because arbitrary same-bank collisions need a protocol contract.

6.7.2 Explicit dual-clock memory

`async_mem<T,N>` is a separate, deliberately narrow semantic resource:

```
memory table : async_mem<u32,1024> {
    write_port wr @write_clk
    read_port rd @read_clk
    init INITIAL_WORD
    read_latency 1
    collision old
    reset { contents preserve read_data clear }
}

table.wr.enable = write_enable
table.wr.address = write_address
table.wr.data = write_data
table.wr.mask = write_mask
table.rd.address = read_address
read_data = table.rd.data
```

It has exactly one write-only port and one read-only port in different explicit domains. Cells change only on the writer edge; `rd.data` changes only on the reader edge and has destination-domain provenance. The contents reset policy belongs to the write domain and the read-result policy to the read domain. Direct use of foreign-domain controls is rejected; a pipeline register is not a CDC primitive.

`async_mem` requires `read_latency` in 1..16. The generic implementation captures the cell on the reader edge and, for latency greater than one, shifts the captured value through reader-domain output registers. This adds visible latency without changing the contents or crossing domains. `read_latency 0` is invalid for `async_mem`; it would be an unregistered cross-clock read, not a block-RAM read port.

For coincident logical edges the simulator uses a pre-edge snapshot: `old` returns the prior cell, while `new` forwards the coincident write. This is a precise digital model, not an analog metastability or silicon timing guarantee. Portable generic Direct-SV publishes `old`; stricter or other collision claims require an exact target capability. Asynchronous 2RW, mixed-width ports, ECC, and dual-clock ROM are not supported.

```
memory table : mem<u8,16> {
    read_latency 1
    collision read_first
}

table.read_address = read_address
```

```

table.write_enable = write_enable
table.write_address = write_address
table.write_data = write_data
read_data = table.read_data

```

Alternatively, one memory can be owned by atomic rules:

```

rule fetch when read_enable {
    table.read(address)
    accepted <- 1
}

rule store when write_enable {
    table.write(address, write_data)
}

```

read registers `table.read_data` one cycle later and it holds when no read fires. `write` commits at the selected edge. One selected read and one selected write may fire together; same-address behavior follows the declared collision mode. Their operands and all other rule effects observe the same pre-edge snapshot. Reset suppresses actions. With the default profile it clears both cells and `read_data`.

Global controls and rule actions cannot be mixed. The bounded scheduled form supports one memory per module, alongside registers and scheduled FIFOs; two reads or two writes conflict and require explicit rule priority.

Executable memory elements may be recursively bit-packable non-enum aggregates, as well as scalars; depth is a power of two and at least two. A globally controlled ordinary `mem` accepts `read_latency` in 0..16: zero is a combinational read; each positive value is an exact number of domain-local read edges from address capture to the visible result. Rule-owned memories remain exactly one-cycle because their read action is selected at an edge. Zero-cycle ordinary `mem` is a generic combinational storage behavior, not a native synchronous BRAM claim. Native target binding requires an explicitly matching clocked-read capability. Until a target mapping proves its internal output-register configuration and any additional fabric stages, a requested latency greater than the advertised native latency uses generic storage under a preferred policy or fails under a required policy.

Cell and visible read-result reset behavior may be selected independently:

```

memory table : mem<u32,256> {
    read_latency 0
    collision read_first
    reset {
        contents preserve
        read_data preserve
    }
}

```

If the `reset` block is present, both directives are required. If it is absent, the exact default is `contents clear` and `read_data clear`. Reset always suppresses writes and scheduled actions. At latency zero, `read_data clear` masks the combinational result while reset is asserted; `read_data preserve` leaves the addressed preserved cell visible. At latency one, the corresponding policy clears or holds the result register. `read_first` observes the old cell before a coincident write edge; `write_first` observes the fully byte-mask-merged write word.

The simulator and generic Direct-SV initialize executable preserved memory state to its declared uniform `init` word, or zero when omitted, but runtime reset does not recreate that initialization under `contents preserve`. The Xilinx same-clock true-dual mapping therefore requires preserved contents; Vivado maps its source initializer to BRAM initialization. Target selection stays fail-closed when a resource does not advertise initialization support. Full/partial writable-memory images, automatic banking, unbounded multiport memory, and asynchronous 2RW are not part of this hardware surface. Bounded named-port selection is described above. A Direct-SV build may publish a Verilator VPI state-access companion with `--simulation-state-bundle`; this does not add hardware ports or change memory reset semantics. The [writable-memory contract](#) and public [simulation-state access](#) sections define the two distinct boundaries.

6.7.3 Replicated read ports, wrappers, and banking

Multiple logical read ports may be built explicitly from coherent replicated 1R1W memories: every replica receives the same write, while each read port uses its own replica. Banking and arbitration remain source-level design choices; the compiler does not choose a bank mapping automatically.

`StorageDualPortMemory`, `Storage2R1WMemory`, and `StorageAsyncMemory1W1R` provide reusable common memory shapes. `StorageAsyncFifo` uses the explicit ready/valid `async_fifo(D)` crossing with independent write/read clocks and a registered reader-side memory path; an ordinary `fifo<T,N>` never becomes asynchronous.

A target-specific memory binding is selected only when its width, depth, clock, latency, reset, and collision capabilities match exactly. Under a `required` policy, an unsupported shape is rejected; under `preferred`, it may remain generic RTL. Automatic banking, byte-addressed external wrappers, and target-specific latency or collision adaptation are not supported.

6.8 Initialized synchronous ROMs

An immutable lookup table is declared separately from writable memory:

```
rom twiddles : rom<Complex<Twiddle>, N / 2> {
    read_latency 1
    init fft_twiddles<T=Twiddle,N=N>()
}

twiddles.read_address = phase
selected_twiddle = twiddles.read_data
```

The initializer must specialize at compile time to exactly `vec<depth,element_type>`. Elements may be bit-packable scalar/fixed values, vectors, or concrete structs; protocol/state values and nominal enums are rejected. Depth is positive and may be one or non-power-of-two. The address is an unsigned `uint<max(1,ceil_log2(depth))>` value whose proven range remains inside the declared depth.

The read is a **one-cycle synchronous read**. During reset the registered read result is zero. Reset never changes the immutable contents; after reset, the first non-reset result corresponds to the preceding accepted non-reset address. There is no hidden second output register.

Direct-SV consumes one generated companion image. It contains one exact-width binary word per line, address zero first. Struct declaration field zero occupies the most-significant bits; vector element zero and tuple item zero occupy the least-significant component bits recursively. Fixed-point values retain their raw signed or unsigned bit pattern. Direct-SV `$readmemb` consumes that image. The CLI publishes companions beside the selected output, and artifact metadata retains the initialization dependency, evaluator, content, and file hashes. Missing or colliding companion files fail closed.

Writable or partial initialization, reloadable/asynchronous/multiport ROM, enum elements, and automatic BRAM/storage exploration are not supported.

6.9 Stateful hierarchy

Single-domain scalar child modules may contain registers and rules. A compile-time indexed instance array may contain bounded scalar sequential or primitive ready/valid children with one matching clock/reset domain; each physical instance has independent state and reset. Aggregate scalar outputs and mixed scalar/ready-valid ports retain their exact typed leaves. Storage-only child arrays may own one FIFO, synchronous memory, or initialized ROM. The scheduled-FIFO profile may also combine FIFO actions with ordinary register writes/rules in one atomic transition; globally controlled storage plus user state remains rejected.

One outer array may contain a bounded same-domain scalar or direct-ready-valid hierarchy, including an inner scalar compile-time array. First-level in-order request/response requester/responder arrays are supported with scalar wire peers and explicit indexed connections. Nested storage/CSR/aggregate protocols, nested or out-of-order request/response, other non-RV protocols, CDC, runtime-selected instances, and cross-module atomic scheduling remain fail-closed. See [Hierarchy and protocols](#) for the complete instance-array boundary.

7 Physical clocks and resets

Each ZLang clock domain defines the complete physical clock/reset contract. Concise declarations retain the default reset contract:

```
clock clk
reset rst @clk
```

normalizes to rising-edge clocking, synchronous active-high reset, native release, and unspecified power-up state.

7.1 Recommended asynchronous reset

The concise asynchronous form is safe for ordinary state. In a multi-clock module each annotated domain owns an independent conditioner:

```
clock clk
async reset arst @clk
```

It means asynchronous assertion followed by deassertion synchronized through two registers. Reset remains active on the first two active clock edges after the external pin is deasserted; normal state transition resumes on the third edge. Reassertion at any point immediately restarts that release sequence.

The external pin may be active-low:

```
clock clk
async reset arst_n @clk {
    polarity active_low
}
```

If reset deassertion is already synchronized to this exact clock outside the generated module, state that physical contract explicitly:

```
clock clk
async reset arst_n @clk {
    polarity active_low
    release externally_synchronized
}
```

This retains asynchronous assertion for resettable state but emits no internal two-register release conditioner. The annotation is never inferred: every destination clock domain must receive a release synchronized to that clock. Omitting `release` retains the safe internal 2FF default.

A register participates in reset only when it declares a reset value:

```
reg control : u8 = 0 // asynchronous reset to zero
reg datapath : u8    // no reset branch or reset sensitivity
```

The unreset register continues to accept enabled clocked updates while reset is asserted. Its RTL power-up value is unspecified until the design writes it. Native two-state simulation uses a deterministic zero startup seed, which is not a source-level power-up or reset guarantee.

A falling-edge clock uses falling edges for the two release cycles as well. Polarity affects the external pin and generated RTL; the simulator still accepts a logical True for asserted reset.

The lower-level compatibility spelling remains available for a raw native asynchronous assertion/deassertion contract:

```
clock clk { edge falling }
reset rst_n @clk {
    mode asynchronous
    polarity active_low
    power_up unspecified
}
```

This full block does not insert synchronized release. When either low-level physical block is present, every clause in that block is required. Raw active-low protocol helpers gate transfers with the deasserted high level; they do not reinterpret the physical pin as active-high.

7.2 Backend lowering

Direct-SV emits a deterministic two-register synchronizer marked with `ASYNC_REG` for the unannotated concise form. The top-level domain owns it and routes one conditioned reset through register/rule, FIFO, memory, CSR, protocol, and child state. A hierarchy does not create one synchronizer per sibling. A clocked but completely state-free module publishes the same physical contract without emitting an unused conditioner, because no reset epoch is consumed.

Within a composed artifact, each child receives the top's conditioned net and uses it directly as its asynchronous reset. Children and nested grandchildren never add another release conditioner, so hierarchy depth does not add reset release latency. The same reusable child compiled as a public top owns its own conditioner, because that compilation has a new external reset boundary.

`release externally_synchronized` routes the external reset directly into the asynchronous event control of resettable state and emits no conditioner. Resetless registers use only the active clock event.

Native simulation models each input item as one active edge. It therefore models immediate assertion at the sampled boundary and the exact two-edge release hold. Assertion between active edges is additionally checked in RTL simulation.

Generated implementation metadata records the external clock/reset paths, edge, assertion mode, polarity, release cycles, power-up policy, and source origin.

7.3 Formal applicability

Executable formal work consumes the exact declared contract. With `power_up unspecified`, the supported combinations are described below.

Formal checks of rule firing use the effective reset and polarity, not a raw active-high input assumption. Descendants consume the conditioned native reset. Rising and falling active edges, active-high and active-low pins, and synchronous or asynchronous assertion are supported. An asynchronous reset may use native or exactly two active edges of synchronized release. Applicable routes are source safety/cover, bindable recursive safety, same-cycle or fixed-latency $\Pi=1$ Direct-SV equivalence, and formal-aware selection. Asynchronous formal execution is limited to one physical domain.

Multiple supported synchronous domains may still produce independent goal-local jobs. Asynchronous execution is deliberately single-domain; this does not define a cross-domain reset relation. Same-cycle pure equivalence remains reset-independent, while fixed-latency comparison uses the declared active edge and release window.

The same clock/reset contract applies to RTL, formal jobs, and their results. Missing, corrupted, or mismatched metadata is rejected rather than interpreted as another reset mode. A non-executable goal still reports its declared contract and the reason that no compatible formal route exists.

Formal traces distinguish the two reset views. `physical_reset` is the raw external pin with its declared polarity. `trace:reset` and result `reset_state` are the normalized active-high *effective* reset used by property guards, previous-cycle history, feasibility covers, fill masks, and comparison windows. For synchronized release, effective reset remains asserted for both release edges even though the external pin is already deasserted. Failure-cycle and originating-sample attribution therefore describe the real reset epoch, not merely the raw pin level.

7.4 Boundaries

One clock domain has exactly one reset. Declaring both `reset` and `async reset` for that domain is an error. Direct-SV supports multiple independently conditioned asynchronous domains; current formal execution still skips an asynchronous goal in a multi-domain module because no cross-domain reset-epoch relation is claimed. Implicit reset crossings, reset combinators, configurable synchronizer depth, glitch filters, power-on reset, and macro-selected RTL semantics are not supported.

`power_up reset` remains typed but fails closed because no common portable synthesizable initialization mechanism is frozen. DSP48 physical reset pins, elastic or variable-latency `pipeline(auto)` equivalence, CDC/reset-refinement proofs, and target BRAM reset pins remain unsupported. The same applies to multi-domain asynchronous formal execution, general hierarchical equivalence, and any route missing a compatible domain, binding,

assumption, or observation. Formal-aware selection records unavailable evidence without changing eligibility; a required policy fails unless the exact equivalence check completes at the requested level.

8 Hierarchy, protocols, and explicit CDC

ZLang represents modules, endpoints, ownership, clock domains, connections, and physical instances explicitly before backend lowering. Connections are never inferred by guessing generated RTL names.

8.1 Modules and instances

Modules may have compile-time type and value parameters:

```
module Engine<type T, DEPTH=4> {
  in  value : T
  out copy  : T = value
}
```

Instantiate and bind scalar inputs explicitly or by same-name shorthand:

```
inst engine : Engine<T=u8,DEPTH=4> { value }
result = engine.copy
```

Specialization identity is distinct from physical instance identity. A single-domain parent may supply an unambiguous child clock/reset implicitly; multi-domain hierarchy requires explicit compatible domains. Child protocol endpoints use `connect`, not scalar binding syntax.

Identical parameterless child components with the same reachable dependencies may share one emitted implementation. Different type/value specializations, or different reachable helper definitions, remain distinct. Helpers merely visible through a parent or import context do not affect this identity.

When every intermediate child conforms to one exact named interface with one protocol input and one protocol output, an option-free path can be written as:

```
input -> decode -> execute -> output
```

This is equivalent to the pairwise typed edges `input -> decode.rx`, `decode.tx -> execute.rx`, and `execute.tx -> output`. Intermediate names are physical instances, not guessed ports. Arrays, unnamed or ambiguous signatures, edge options, adapters, buffering, and crossings remain explicit.

One-dimensional compile-time instance arrays are structurally unrolled:

```
inst lane[N] : Lane<W>
generate(i in 0..N) {
  lane[i].x = values[i]
}
outputs = generate(i in 0..N) lane[i].y
```

A runtime selector may project one exact bit-packable wire output. This is a read-only mux over the already elaborated children, never a dynamic physical instance:

```
selected = lane[select].output
```

Every `lane[i]` still exists, receives its compile-time bindings, and advances its own state independently. The selector neither routes child inputs nor gates a child clock, reset, rule, or storage action. The explicit equivalent is `outputs = generate(i in 0..N) lane[i].output` followed by `selected = outputs[select]`; both spellings retain the same physical child identities.

The bounded supported form covers:

- combinational scalar children, including exact aggregate scalar outputs;
- single-domain scalar children with registers/rules;
- primitive ready/valid children, including scalar wire ports beside the endpoint and the one-FIFO profile;
- storage-only children owning one FIFO, synchronous memory, or initialized ROM;

- the scheduled-FIFO profile in which FIFO actions and ordinary register writes share one atomic transition; and
- an outer one-dimensional array whose element contains a bounded same-domain scalar or direct-ready-valid hierarchy. Inner compile-time scalar arrays are retained as distinct physical paths rather than flattened by generated name.

The first-level request/response profile is also bounded and explicit. Each array child has exactly one `ordering` `in_order` endpoint plus scalar wire ports, and each connection names compile-time indexed requester and responder elements:

```
generate(i in 0..N) {
    requester[i].issue = issue[i]
    responder[i].accept = accept[i]
    connect requester[i].mem -> responder[i].mem
}
```

Every physical element retains its own outstanding ledger and reset epoch. Out-of-order matching, an additional protocol on the same child, nested or transitive request/response arrays, and unindexed endpoint references are unsupported.

8.1.1 Storage-owning instance arrays

A bounded one-dimensional instance array may contain same-domain scalar-wire children that own at most one FIFO, synchronous memory, or initialized ROM. Every element has independent state and one shared specialization. The globally controlled storage form remains storage-only; the scheduled-FIFO form may combine FIFO actions with ordinary register/rule state in one atomic transition.

Runtime selection is limited to a read-only projection of a bit-packable scalar-wire output. It is an output mux over statically elaborated children and never gates a child or creates a dynamic instance. Storage arrays require one inherited clock/reset domain and no nested instances. Multiple storage resources, CSR/storage/protocol mixtures, CDC, runtime-selected inputs or protocol endpoints, and ready/valid arrays containing memories or ROMs are not supported. A storage-only child with one globally controlled FIFO may expose primitive ready/valid ports.

Credit and other non-RV protocols, storage below a nested array element, cross-module atomic scheduling, and incomplete bindings are rejected.

Concise declarations have the same hierarchy meaning when unambiguous:

```
frontend : AXI4LiteToRegBus<32,32>
axi : AXI4Lite<32,32>.slave @clk
axi -> frontend.axi
```

Explicit `inst` remains the escape hatch for namespace ambiguity.

8.1.2 Typed external modules

A bounded scalar-wire component may declare backend-independent behavior with an ordinary pure ZLang model:

```
extern module VendorAdd : AddIfc { model add_model }
```

The concise source form is `extern module VendorAdd : AddIfc { model add_model }`.

The applied interface and model produce one exact semantic signature. Physical SystemVerilog is supplied separately through a hash-validated external physical mapping declared under `[external-mappings.NAME]` in `zlang.toml` and selected by a profile. Inline HDL never defines ZLang semantics. The supported form has non-parameterized, clockless scalar inputs and one scalar output in Direct-SV. State, protocols, arrays, and generic external components fail closed.

8.2 Clock and reset boundary

`clock clk` plus synchronous active-high `reset rst` works through the production Direct-SV backend. A single-domain parent may supply that exact domain to a child only when the mapping is unambiguous; this is domain

inheritance, not implicit CDC. One non-default falling-edge, asynchronous, or active-low physical contract is typed and emitted by Direct-SV. Concise `async reset` adds one root-owned two-edge release conditioner and passes the conditioned reset through the closed child ABI; the raw asynchronous compatibility form remains distinct. Instance arrays require one compatible inherited domain unless they are purely combinational. Multiple independently conditioned asynchronous domains are supported in Direct-SV. Every domain of a stateful child must match exactly one parent physical clock/reset contract; no implicit reset crossing is inserted. Multi-domain asynchronous formal execution, power-on reset, and implicit reset crossing remain fail-closed.

8.3 Public top ABI

The selected top always exposes integration-friendly typed leaves. Struct fields and structural tuple components, including those inside protocol payloads, become individually named ports; tuple paths use `item0`, `item1`, and so on. A vector leaf remains one multidimensional packed SystemVerilog array; for example `vec<8,u8>` becomes `logic [7:0][7:0] samples`. It is neither an anonymous 64-bit public bus nor eight separately named ports. Nested `vec<Struct>` values become one array per struct field, and `vec<Tuple>` values become one array per tuple component. A tagged-union port uses one named packed structure containing its exact `tag` and `payload` fields.

Direct-SV uses this public top-level port ABI. The conversion follows the canonical packing order with element zero in the least-significant region: for a descending packed vector dimension, source element zero maps to physical index 0. There is no source annotation, compiler option, or backend-specific preprocessor branch selecting another public ABI.

8.4 Ready/valid

```
in  rx : rv<u8>
out tx : rv<u8>

tx.payload = rx.payload
tx.valid   = rx.valid
rx.ready   = tx.ready
```

For an input endpoint, `payload/valid` enter the module and `ready` leaves it. Output ownership is reversed. `.transfer` is a read-only protocol property meaning `valid & ready`. A source stalled with `valid=1` and `ready=0` must hold `payload` and `valid` stable.

Direct composition names source then sink:

```
connect rx -> tx
connect buffered_rx -> buffered_tx { buffer 2 }
```

A `ready/valid` buffer is real FIFO state; it does not appear implicitly.

8.5 Credit and request/response

`credit<T,N>` provides pulse-return flow control. `.transfer` is the physical send after credit gating, `.credits` is the current bounded count, and `.return` returns one credit. Counters begin at `N`.

Adapters are explicit:

```
connect rx -> tx { adapter rv_to_credit }
connect crx -> rtx { buffer 2 adapter credit_to_rv }
```

Request/response endpoints declare ordering and outstanding capacity:

```
interface mem : request_response<Request,Response> {
    max_outstanding 2
    ordering in_order
}
```

The requester owns request `payload/valid` and response `ready`; the responder owns request `ready` and response `payload/valid`. Request and response channels expose their own read-only `.transfer` events. Directional buffers are independent:

```
connect requester.mem -> responder.mem {
    request_buffer 4
    response_buffer 2
}
```

Buffered but unaccepted requests are distinct from accepted outstanding requests. The ledger increments on responder-side request transfer and decrements on requester-side response transfer. Reset begins a new protocol epoch. `ordering out_of_order` requires a matching scalar ID field; hierarchical out-of-order composition remains outside the current bounded subset.

8.6 Aggregate protocols and the standard library

The built-in `std` namespace contains ordinary ZLang HDL modules. Aggregate protocol identity, roles, ownership, hierarchy, and clock domains remain language-level properties; AXI/APB transaction behavior stays in library modules rather than being hard-coded into the language.

Available source-authored profiles include RegBus, AXI4-Lite, APB, AXI-Stream, Wishbone B4 Classic, bridges, and the CSR target:

```
import std.bus.reg
import std.bus.axi_lite

module AxiCsrTop {
    clock clk
    reset rst
    interface axi : AXI4Lite<32,32>.slave @clk
    inst frontend : AXI4LiteToRegBus<32,32>
    inst csr : RegBusCSRTarget<32,32>
    connect axi -> frontend.axi
    connect frontend.regbus -> csr.regbus
}
```

Aggregate pass-through connects each member according to typed ownership, including reverse ready:

```
input -> output
```

Protocol member properties such as `.transfer`, FIFO observations, and request/response channel events are part of typed semantics. They are not arbitrary user-defined fields and cannot be assigned when read-only.

8.7 Packets, virtual channels, and arbitration

`packet<T>` extends `ready/valid` with `.last`. Arbiters explicitly select source order, `fixed_priority` or `round_robin`, and `beat` or `packet` grant lifetime. `vc_credit<T,V,C>` maintains independent credit counts per virtual channel.

8.8 CSR blocks

CSR source describes address layout, access policy, reset, and hardware binding once. Supported policies are `rw`, `ro`, `wo`, `w1c`, `pulse`, and `reserved`. Hardware status uses `<-`, commands use `->`, and sticky events make simultaneous hardware/software priority explicit. RTL, JSON, and Markdown are derived from the same typed model.

Status and sticky sources may select exact leaves of a typed aggregate input; the compiler retains and type-checks the member expression rather than deriving a flattened name:

```
struct Perf { cycles : u32 }
struct EngineStatus { busy : bit perf : Perf }

module AggregateStatusBank {
    clock clk reset rst
    in status : EngineStatus
    csr registers @0 {
        STATUS @0 { busy bit @0 ro <- status.busy }
        CYCLES @4 { value u32 @31:0 ro <- status.perf.cycles }
    }
```

```

    }
}

```

Aggregate members used by one CSR bank must belong to that bank's clock domain. Command bindings remain scalar output ports.

Omitted bits are implicitly reserved. A storage-independent access event may observe the same bits as their single owning field:

```

module CsrAccessEventExample {
    clock clk reset rst
    in fault_status : bits<32>
    out fault_clear : bits<2>
    csr registers @0 {
        FAULT @0x470 {
            value bits<32> @31:0 ro <- fault_status
            clear bits<2> @1:0 on_write -> fault_clear
        }
    }
}

```

The event is address-qualified by the compiler and owns no state. Stored, external-status, and event observations are available through typed child namespaces `state`, `status`, and `events`.

Repeated layouts flatten into one bank and one decoder:

```

module CsrGroupExample {
    clock clk reset rst
    csr group Window { CONTROL @0 { enable bit @0 rw = 0 } }
    csr control @0 { windows : Window[13] @0x210 stride 4 }
}

```

Group counts are bounded to 1..64 and a bank to 256 expanded registers. A logical 64-bit value may retain two independently writable 32-bit software registers:

```

module CsrSplitExample {
    clock clk reset rst
    csr registers @0 {
        INPUT_BASE @0x18 split<32> value u64 rw = 0 order low_first
    }
}

```

The same `split<32>` declaration may be part of a reusable group. Every group instance receives one logical typed projection and two independently writable 32-bit words:

```

module GroupedSplitExample {
    clock clk reset rst
    csr group Window {
        CONTROL @0 { enable bit @0 rw = 0 }
        BASE @4 split<32> value u64 rw = 0 order low_first
    }
    csr registers @0 { windows : Window[2] @0 stride 12 }
}

```

This does not imply byte strobes or an atomic two-word write. Typed command capture and registered RegBus timing are ordinary source modules: `StorageSnapshot<T,reset_value>` in `std.storage.core` and `RegisteredCSRTarget<AW,DW>` in `std.bus.reg`; the compiler has no special case for either module name.

In a multi-clock module the bank declares its owner after its base address:

```

csr control @0x4000_0000 @apb_clk { /* registers */ }

```

All blocks sharing the canonical access ABI must use that one domain. A runtime CSR value consumed by another domain still requires explicit CDC.

8.9 Clock-domain crossings

Implicit CDC is an error. Supported explicit forms are:

Crossing	Supported endpoint
<code>sync_level</code>	Slowly changing bit level
<code>pulse_toggle</code>	Rate-limited bit pulse
<code>handshake</code>	One scalar ready/valid payload
<code>async_fifo(N)</code>	Ready/valid stream through a power-of-two dual-clock FIFO

```
connect source -> destination { crossing async_fifo(4) }
```

The crossing is a semantic boundary. Its output has destination-domain provenance and may feed destination-domain combinational logic, rules, FSMs, or state normally. The compiler never rewrites, balances, or resource-packs logic through the boundary, and never chooses a crossing automatically.

```
connect enable_a -> enable_b { crossing sync_level }
rule Observe @clk_b when enable_b { seen <- 1 }
```

An aggregate with exactly one forward ready/valid member may use the same `async_fifo` crossing; its complete payload is atomic. Both domains must participate in a coordinated reset episode.

The physical Direct-SV FIFO uses one 1W1R `async_mem` with a one-cycle registered read, two-stage Gray-pointer synchronizers, and one prefetched output beat. A stalled beat holds both payload and `valid`; the read/consumed pointer advances only on transfer, so that beat still occupies capacity. Continuous accepted beats can transfer at $\Pi=1$ without an extra output bubble. The synchronizers model digital CDC behavior, not analog metastability or an MTBF guarantee.

The ordinary `std.storage.core.StorageAsyncFifo<T,D>` wrapper exposes this same crossing as a reusable module with explicit writer and reader clock/reset ports. It uses the explicit `async_fifo` CDC contract. A normal `fifo<T,N>` never changes into an asynchronous FIFO because its endpoints happen to use different domains.

No automatic protocol adaptation or implicit CDC is performed. Full AXI4 bursts/IDs and bus-specific CDC bridges are not supported.

9 Named module interfaces

Named module interfaces give a public name to an exact, behavior-free module signature. They let a library state the ABI that several concrete modules must implement without selecting, replacing, or instantiating any implementation.

```
interface FirIfc {
    clock clk
    reset rst
    in x : fixed<18,16>
    out y : fixed<18,16>

    timing {
        latency 4
        ii 1
    }
}

module DirectFir : FirIfc {
    // The complete public surface is inherited from FirIfc.
    y = pipeline(4) { x }
}
```

The declaration after `interface` contains only public signature items:

- type and value parameters;
- clock and reset declarations;
- scalar, wire, ready/valid, credit, packet, and VC-credit in/out ports;
- source-authored aggregate protocol endpoints and their roles;
- at most one exact timing { latency N ii 1 } contract.

It cannot contain assignments, registers, rules, storage, child instances, or other implementation behavior.

9.1 Exact conformance

`module M : Ifc` means that `M` promises to have exactly the applied signature of `Ifc`. Conformance is checked during typed semantic analysis. It is not structural duck typing and does not introduce an implicit conversion.

The following all belong to the contract:

- parameter names, kinds, declaration order, and defaults;
- applied type and value parameter values after specialization;
- port names, declaration order, directions, canonical types, and protocol capacities;
- clock/reset pairs and every port or endpoint domain;
- aggregate protocol specialization, role, member ownership, payload types, and member domains;
- presence and exact value of the module timing contract.

If a conforming module declares no public surface, the complete applied interface surface is inherited: parameters, ports, clock/reset domains, aggregate endpoints, and timing. This is exact signature inheritance, not structural inference. The fully redeclared form is also accepted. A partial redeclaration is never merged with the interface and fails exact conformance.

Missing, additional, or renamed members in a fully redeclared surface are errors, as are changed directions, widths, roles, domains, or timing contracts.

Named parameters and concrete type parameters are supported:

```
interface TransformIfc<type T, N=4> {
  in  x : vec<N,T>
  out y : vec<N,T>
}

module Transform<type T, N=4> : TransformIfc<T=T, N=N> {
  y = x
}
```

The module parameter declaration itself must match the interface parameter declaration exactly. A concrete specialization records the canonical applied type and value parameters; concise source spelling does not change its interface identity.

9.2 Aggregate protocol signatures

A source-authored aggregate protocol can be part of a named module interface. Its role and clock domain are explicit:

```
protocol TinyBus<AW=8> {
  role initiator
  role target
  channel command : rv<bits<AW>> initiator -> target
  member alarm : bit target -> initiator
}

interface TinyTarget<AW=8> {
  clock clk
  reset rst
  interface bus : TinyBus<AW=AW>.target @clk
}
```

```

module Target<AW=8> : TinyTarget<AW=AW> {
  clock clk
  reset rst
  interface bus : TinyBus<AW=AW>.target @clk
  // Concrete behavior remains in the module.
}

```

Conformance uses the already typed aggregate schema and ownership model. It does not guess compatibility from endpoint or generated RTL names.

9.3 Selection and backend identity

A named interface never chooses a concrete implementation. Source still instantiates a concrete module, and normal top selection still names a concrete module. There is no automatic substitution, inheritance, refinement search, or runtime dispatch.

The exact applied interface specialization is preserved through compilation and Direct-SV generation. Merely adding an equivalent interface declaration does not authorize a backend to change RTL or timing.

9.4 Boundaries

The bounded surface intentionally does not support:

- `request_response` members in named module interfaces, because their requester/responder role is currently inferred from module behavior rather than declared as an exact signature role;
- interface extension or refinement beyond exact complete-surface inheritance;
- multiple-interface conformance;
- automatic implementation selection or substitution;
- relaxed variance, implicit resizing, protocol adaptation, or CDC insertion.

Use the concrete module and protocol forms when one of these boundaries applies. Unsupported or non-conforming signatures fail with a structured interface diagnostic; they never silently weaken the ABI.

10 Tagged unions

ZLang tagged unions are a bounded, backend-independent way to carry one of several named scalar payload shapes. They are values, not procedural control flow and not protocol envelopes.

```

union Message {
  Idle
  Data { value : u8 }
  Error { code : bits<4> }
}

message : Message = Message.Data { value = input }

result : u8 = match message {
  Message.Idle => 0
  Message.Data { value } => value
  Message.Error { code } => extend<8>(code)
}

```

10.1 Type and layout

The type is nominal: two equal-looking declarations are not interchangeable. Variant order defines ordinal tag codes. The tag width is $\max(1, \text{ceil_log2}(\text{variant_count}))$ and occupies the most-significant bits. The payload is as wide as the largest variant. Fields occupy it from most to least significant in declaration order; a shorter payload has zero low-order padding.

The supported field types are flat `bit`, `bits<N>`, `uN/uint<N>`, `sN/sint<N>`, `fixed`, and `ufixed` fields. A fieldless value uses the concise constructor `Message.Idle`; `Message.Idle {}` remains an equivalent explicit spelling.

10.2 Construction and matching

Constructor fields are named, exact, and complete. Missing, extra, repeated, or wrongly typed fields are compile-time errors. A `match` must contain every variant exactly once. A payload arm binds every field in declaration order; binders cannot rename, omit, repeat, or shadow an existing value.

Matching is pure and zero-latency. A tagged union keeps the exact declared tag and payload layout through native simulation and Direct-SV generation. Registers and module inputs/outputs may use union values. Direct-SV exposes a selected top's tagged-union port through a named packed structure containing the exact tag and payload fields; its internal representation remains the same frozen packed layout.

10.3 Deliberate boundaries

An external producer of a tagged-union input must drive one of the declared tag codes; unused raw tag codes are outside the module contract. There is no raw decode, `bitcast<Union>`, `unpack<Union>`, generic/recursive union, nested aggregate payload, union operator overload, wildcard/nested pattern, guard, partial match, or protocol inference. Tagged unions have no dedicated formal checks.

The runnable source is `examples/tagged_union.zhl`. The exact representation and exclusions are documented above and in the [syntax support matrix](#).

11 Parameterized aggregate protocols

A parameterized protocol endpoint is specialized from its compile-time arguments. For example:

```
protocol TinyBus<AW=8> {
  role initiator
  role target
  channel req : rv<uint<AW>> initiator -> target
  channel rsp : rv<uint<AW>> target -> initiator
  member irq : bit target -> initiator
}
```

A module uses this declaration with `interface bus : TinyBus<AW=8>.initiator`. Aggregate protocol identity, specialization, role, member ownership, payload types, and clock domains are preserved through hierarchy and Direct-SV generation. Ready/valid members preserve physical backward-ready propagation; plain members are ordinary scalar wires.

Top-level aggregate endpoints are exposed through deterministic typed physical leaves according to their source ownership. Aggregate and member identities remain separate from generated names.

`connect left.bus -> right.bus` checks protocol identity, specialization, member names, payload types, roles, and clock domains before producing leaf hierarchical connections. Aggregate buffering, adapters, and CDC are deliberately rejected; they must be expressed on a leaf connection in a later library design.

Across child boundaries, every scalar dependency and protocol backward signal is explicit, and forward values and scalar outputs return through equally explicit connections. No aggregate or RTL name is reconstructed by textual substitution. Implementation manifests publish both aggregate identities and their leaf signal bindings for formal and source attribution.

Generic value parameters support exact positive width arithmetic (+, -, *, and exact /). Type parameters are bound at an aggregate use site. The aggregate connection is structural: it does not implement AXI/APB transaction state machines, adapters, packages, CDC, or automatic protocol conversion.

The TinyBus producer/consumer example (two ready/valid channels plus a reverse scalar member) elaborates into three typed leaf connections.

Stateful children with multiple independent ready/valid channels are supported in Direct-SV. An unconnected aggregate bus on the selected top is mapped into typed public leaves. This is generic aggregate lowering, not an AXI semantic exception. Arrays, partial aggregate exposure, and unsupported protocol kinds remain unsupported as listed in the live [syntax matrix](#).

12 Standard buses

The production bus profiles are source-authored standard-library modules; importing a bus does not change language semantics.

Import	Source-owned declarations	Support boundary
<code>std.bus.reg</code>	<code>RegBus</code> , CSR bank/target	In-order request/response CSR boundary
<code>std.bus.axi_lite</code>	<code>AXI4Lite</code> , <code>AXI4LiteToRegBus</code>	32/32-compatible, independent AW/W buffering
<code>std.bus.axi_burst</code>	<code>AXI4BurstSubset</code> , read/write views and helpers	No-ID, single-outstanding, full-width incrementing bursts
<code>std.bus.axi4</code>	Five-channel <code>AXI4</code> / <code>AXI4WithUser</code> , bounded read/write managers	Bounded IDs, bursts and optional USER payloads
<code>std.bus.axi4_subordinate</code>	Bounded read/write subordinate adapters	Bounded transactions and responses
<code>std.bus.axi4_pins</code>	Flat-pin adapters	Source-owned five-channel pin projection
<code>std.bus.axi4_exclusive</code>	Reservation monitor and exclusive subordinate	Bounded explicit same-edge commit contract
<code>std.bus.apb</code>	<code>APB</code> , <code>APBToRegBus</code>	APB setup/access sequencing
<code>std.bus.ahb_lite</code>	<code>AHBLite</code> , <code>AHBLiteToRegBus</code>	Single-manager, full-width AHB-Lite to RegBus
<code>std.bus.axi_stream</code>	<code>AXIStream</code> , <code>AXIStreamPipe</code>	Data/keep/strb/last ready/valid stream
<code>std.bus.wishbone</code>	<code>Wishbone</code> , <code>WishboneToRegBus</code>	B4 Classic single-beat, ack/err/stall

Error completion is part of the source profile rather than a backend policy. AXI4-Lite maps a failed RegBus write or read to `SLVERR` (2'b10) and holds the complete B/R payload while the corresponding ready signal is low. APB forwards the held RegBus error through `PSLVERR` on the access completion. Wishbone uses mutually exclusive normal ACK and abnormal ERR termination; either one retires the single outstanding request.

AHB-Lite preserves the protocol's pipelined address/data relationship. The bridge captures an accepted address/control phase, consumes write data in the following data phase, and holds `HREADYOUT` low while its one RegBus request is pending. Local size/alignment errors and RegBus failures both use the standard two-cycle ERROR response: low-ready/high-response followed by high-ready/high-response. The first profile accepts aligned, full-bus-width transfers for power-of-two byte-addressable data widths from 8 through 1024; subword transfer strobes are not supported.

The bridge's public domain is active-low `hresetn` with asynchronous assertion and the language's two-edge synchronized release. The external reset pin keeps its AHB polarity, `HREADYOUT` is high during reset, and connected sequential RegBus logic must share that exact physical reset contract.

Standard-library module identity and content participate in build identity, so changing a used library source invalidates dependent artifacts and evidence. Imports resolve through the built-in `std` namespace; cycles and unsafe paths are rejected.

The AXI4 source modules include IDs, bounded outstanding slots, five independent channels, and optional USER sidebands. General drop-in AXI4 endpoint compliance is outside the alpha contract; the supported behavior is the bounded source model documented here. The ordinary subordinate never emits `EXOKAY`; exclusive commit requires the separate `axi4_exclusive` contract and same-edge `exclusive_grant` gating. The narrower `axi_burst` profile is distinct. AXI-Stream ID/dest/user, Wishbone burst/retry, automatic adapters, and CDC are not added by these profiles.

Runnable integrated examples are [AXI4-Lite CSR](#), [APB CSR](#), [AHB-Lite CSR](#), [Wishbone CSR](#), and the [streaming packet engine](#). The bounded burst helpers are demonstrated by the [AXI burst example](#).

The AHB-Lite contract follows the [Arm AMBA 3 AHB-Lite protocol](#) with its pipelined address and data phases. Write address/control and `HWDATA` are not sampled in the same phase, and ERROR is not shortened to one cycle. That bounded contract is normative; broader AHB features remain explicit exclusions below.

12.1 Bounded AXI burst subset

`std.bus.axi_burst` is ordinary source-authored ZLang HDL. It defines a combined `AXI4BurstSubset<AW,DW>` with independent AR/R and AW/W/B ready/valid channels, together with read-only and write-only protocol views. Address payloads carry byte address, eight-bit beats-minus-one `len`, and three-bit beat `size`; read and write data carry `last`, and R/B carry the exact two-bit response value.

The reusable `AXI4BurstReader<AW,DW,LW>` and `AXI4BurstWriter<AW,DW,LW>` accept aligned 1-256-beat requests and permit one outstanding transaction each. AR and AW payloads are snapshotted and remain stable until accepted. Read consumer backpressure drives `RREADY` directly. The writer accepts AW before exposing W, while AW, W, and B remain independent handshakes; its ready/valid producer retains W data while stalled. Counted framing owns completion: the reader reports early or missing `RLAST` but drains the requested beat count, while the writer generates `WLAST` on its locally counted final beat. Any nonzero `RRESP` or `BRESP` sets a sticky boolean error for that transaction epoch.

The `AW=64,DW=32` reader/writer profile supports 1- to 256-beat requests, rejects lengths outside that range and misaligned requests, preserves payloads under independent channel stalls, reports `RLAST/RRESP/BRESP` failures, resets active transactions, and ignores a new start while busy.

This profile deliberately omits IDs, `WSTRB`, burst-kind, lock, cache, protection, QoS, region and user fields, multiple outstanding transactions, UB-DMA burst chunking, command-descriptor decoding, fences, CDC, or implicit adaptation. Its accepted semantic boundary is the bounded profile stated above.

12.2 AXI4-Lite, APB, and RegBus

`std.bus.reg`, `std.bus.axi_lite`, and `std.bus.apb` are ordinary imported library sources. The compiler checks structural protocol roles, typed ready/valid members, ownership, and domains; transaction state machines belong to library modules. `RegBus<32,32>` is the typed CSR boundary.

The AXI4-Lite and APB profiles use one clock/reset and one outstanding operation. `Axi4LiteToRegBus` buffers AW and W independently and joins them after both transfers; AR returns one held R response. `ApbToRegBus` implements setup/access phases and holds controls stable while waiting for `PREADY`. Top-level aggregate endpoints expose typed public leaves (for example, `axi_aw_valid` and `apb_psel`); connect `axi -> frontend.axi` delegates a same-role endpoint to a child.

`RegBusCSRTarget` is ordinary source-authored state with RW, sticky `W1C`, and one-cycle pulse registers. Its response data and valid bit are held until a response transfer. Formal attachment requires exact published observations: properties needing hidden or unsupported state are explicitly skipped or rejected under the selected policy. Full AXI4 is outside these library profiles; the separate bounded AXI burst subset is described above.

13 Standard library

`std` is the built-in standard-library namespace. Its modules are ordinary ZLang HDL sources and obey the same language semantics as user modules; it is not a Python or user-package import, and importing one of its modules does not add implicit adaptation or special behavior.

Import	Purpose
<code>std.math.fixed</code>	Pure exact-width/inferred fixed-point abs, clamp, min/max, MAC and explicit quantization helpers, plus compatibility components
<code>std.math.complex</code>	Generic <code>Complex<T>/Butterfly<S,D></code> , nominal operators, explicit complex quantization
<code>std.math.complex_fixed_18_16</code>	Compatibility Q2.16 complex multiply profile
<code>std.stream.core</code>	Generic <code>FrameBeat<T,M></code> , ready/valid register slice, skid buffer, and FIFO wrapper
<code>std.stream.serialization</code>	Bounded power-of-two vector serializer and raw bit collector
<code>std.stream.complex_fixed</code>	Compatibility Q5.32-to-Q2.16 complex stream quantizer
<code>std.dsp.fft</code>	Exact FFT butterfly, bit reversal, and compile-time twiddle helpers
<code>std.storage / std.storage.core</code>	Bounded delay/FIFO plus raw-bit reorder and ordered ping-pong storage wrappers
<code>std.coding / std.coding.core</code>	Parity, bit reversal, checked polynomial-tap LFSR step, and exact convolution helpers

Import	Purpose
<code>std.bus.reg</code>	RegBus and the source-authored CSR target
<code>std.bus.axi_lite</code>	AXI4-Lite protocol and RegBus frontend
<code>std.bus.axi_burst</code>	Bounded no-ID AXI burst profile, read/write views, and single-outstanding burst/single-beat helpers
<code>std.bus.apb</code>	APB protocol and RegBus frontend
<code>std.bus.ahb_lite</code>	Standards-correct bounded AHB-Lite protocol and RegBus frontend
<code>std.bus.axi_stream</code>	AXI4-Stream beat/profile and backpressure-preserving pipe
<code>std.bus.wishbone</code>	Wishbone B4 Classic and RegBus frontend
<code>std.target.generic</code>	Resource-free generic target identity
<code>std.target.asic.generic</code>	Generic ASIC cell/resource capability profile
<code>std.target.asic.sky130</code>	SKY130 <code>sky130_fd_sc_hd</code> standard-cell synthesis profile
<code>std.target.intel.cyclone_v</code>	Bounded Cyclone-V target/resource inventory
<code>std.target.xilinx.series7</code>	Bounded source-described Series-7 DSP48E1 profile
<code>std.target.xilinx.xc7z030</code>	XC7Z030 part and bounded resource inventory
<code>std.arch.xilinx7_fir</code>	Manual four-resource symmetric-FIR cascade template
<code>std.arch.xilinx7_memory</code>	Bounded Series-7 storage/resource mapping descriptions
<code>std.arch.xilinx7_signed_product</code>	Signed product-reduction architecture descriptions

Imports are resolved transitively, cycles are rejected, and logical path plus SHA-256 content hash participate in build identity. External path/Git packages use the separate pinned `zlang.toml/zlang.lock` project resolver. Filesystem-relative source imports and implicit network lookup are not part of the built-in `std` resolver, and ordinary compilation never fetches dependencies.

`std.bus.axi_burst` publishes a bounded interoperability profile with the combined five-channel `AXI4BurstSubset` <AW,DW> plus read-only and write-only views. `AXI4BurstReader` and `AXI4BurstWriter` implement one full-width incrementing transaction at a time, with 1-256-beat counting, independent channel backpressure, checked `RLAST`, counted `WLAST`, and deterministic boolean error latching for nonzero `RRESP/BRESP`. IDs, write strobes, burst-kind and other full-AXI sidebands, multiple outstanding transactions, UB-DMA chunking, and fences remain outside this bounded profile.

For one full-width beat, `axi_single_beat_address`<AW,DW> constructs `AxiBurstAddress`<AW> with `len=0` and the matching `size`, plus a validity bit for geometry and alignment. `AXI4SingleBeatReader` preserves the burst reader's one-beat `RLAST/RRESP` checks; `AXI4SingleBeatWriter` snapshots AW and W payloads and allows either channel to transfer first. Both retain the one-outstanding busy/done/error convention. See `examples/axi_single_beat.zhl`. These helpers do not add IDs, `WSTRB`, or implicit bus adaptation.

`std.bus.ahb_lite` follows the AHB-Lite address/data pipeline and two-cycle ERROR response. `AHBLiteToRegBus` <AW,DW> accepts one aligned full-width beat at a time for byte-addressable power-of-two DW from 8 through 1024, supplies an all-byte RegBus write mask, and holds the AHB data phase through RegBus request/response stalls. Subword transfers, arbitration, SPLIT/RETRY and burst operation are not part of this profile. Its `hresetn` domain uses active-low asynchronous assertion and two-edge synchronized release; the example carries that exact contract through the CSR hierarchy.

`std.math.complex` has no bus or stream dependency and uses generic structs, functions, and nominal operator declarations. Mixed fixed-point multiplication retains its full exact width and scale, and the caller places every quantization boundary explicitly. In particular, the core contains no implicit `fixed<18,16>` butterfly. The Q2.16 component and stream profiles live in separate compatibility imports `std.math.complex_fixed_18_16` and `std.stream.complex_fixed`; importing numerical Complex arithmetic does not select either profile or import a bus.

13.1 Fixed-point math

`std.math.fixed` provides pure generic `fixed_abs`, `fixed_min`, `fixed_max`, `fixed_clamp`, and exact `fixed_mac` functions plus explicit rounding/overflow helpers such as `fixed_quantize_nearest_even_saturate`<T>. `fixed_abs` widens according to ordinary unary-negation rules, so the most-negative input has an exact positive result; `fixed_mac` preserves the full product and addition widths. The library also retains the source-authored `FixedAbs`, `FixedClamp`, `FixedMinMax`, `FixedMAC`, `FixedSaturatingAdd`, and `UFixedSaturatingAdd` components. They operate on canonical fixed-point types; concise `SF8.8`, `SF_Sat8.8`, `UF8.8`, and `UF_Sat8.8` forms are language aliases rather than library implementations. Encoding and conversion rules are specified in `fixed-point-types.md`.

13.2 Complex values, streams, storage, and coding

`std.math.complex` provides the ordinary parameterized value type `Complex<T>` with `re` and `im` fields. `butterfly_quantized<T>` requires the output numeric profile explicitly; there is no profile-selecting short spelling in the core.

`std.bus.axi_stream` keeps `AXIStream<DW>` as the byte-lane bus profile and also provides `AXIStreamOf<T>` for a semantic payload type:

```
input  : AXIStreamOf<Complex<fixed<18,16>>>.sink @clk
output : AXIStreamOf<Complex<fixed<18,16>>>.source @clk
```

`AXIStream<32>` has `data/keep/strb/last`; `AXIStreamOf<T>` transports exactly one typed `T` per transfer. It does not by itself specify an external AXI TDATA serialization for arbitrary `T`.

Generic stream composition uses `std.stream.core`, independently of AXI:

```
import std.stream.core

struct Meta { tag : u2 }
type Beat = FrameBeat<u8,Meta>

module Queue {
  clock clk
  reset rst
  in input : rv<Beat>
  out output : rv<Beat>

  inst storage : RvFifo<T=Beat,D=4>
  connect input -> storage.input
  connect storage.output -> output
}
```

`RvRegisterSlice<T>`, `RvSkidBuffer<T>`, and `RvFifo<T,D>` reuse the exact language FIFO semantics, including simultaneous push/pop and payload stability under stall. `std.stream.serialization` supports a power-of-two `RvVectorSerializer<T,N,IW>` and a raw `RvBitCollector<N,IW>`, where `IW = floor_log2(N)`. Their declarations enforce `N >= 2`, power-of-two `N`, and exact `IW` through compile-time `where` constraints.

`std.dsp.fft` keeps arithmetic intent explicit. Its butterfly does not quantize, twiddle generation uses compile-time `sin/cos` and quantizes only to its caller-selected type, and bit reversal is a deterministic sequence-order operation. Value parameters inferred through a `bits<N>` argument are not implemented yet, so bit-width helpers use explicit calls such as `fft_bit_reverse<N=8>(value)`.

`std.storage.core` supplies `StorageDelay1<T>`, `StorageDelay2<T>`, the generic `StorageQueue<T,D,CW>` (`CW = ceil_log2(D + 1)`), and bounded raw-bit `StorageReorderBits<W,N>/StoragePingPongBits<W,N>` banks. The latter use power-of-two depth, exact runtime indices, and explicit bitcast at typed/raw boundaries. Ping-pong publication is ordered: a second commit is blocked until the visible read bank retires, while simultaneous retire/-commit is legal. The same file provides ordinary `StorageDualPortMemory<T,N,AW>`, `Storage2R1WMemory<T,N,AW>`, and `StorageAsyncMemory1W1R<T,N,AW>` wrappers over the same typed memory semantics. `StorageAsyncFifo<T,D>` wraps the explicit ready/valid `async_fifo(D)` crossing and does not authorize implicit CDC. It also provides immutable, source-authored generic ROM wrappers:

```
inst direct : StorageRom<T=u8,N=8,IW=3,image=image>
inst generated : StorageGeneratedRom<
  T=u8,N=8,IW=3,producer=fn make_image<T=u8,N=8>
>
```

Both wrappers produce the same immutable ROM contents and deterministic companion image used by Direct-SV. `StorageRom` accepts a fully evaluated exact `vec<N,T>` constant; `StorageGeneratedRom` invokes a statically selected pure zero-argument producer during elaboration. Constants and producers are specialization parameters, not runtime ports or backend callbacks. Fixed, struct, vector, and nested-vector word types retain their exact canonical layout.

The core language requires a literal in `delay<N>`, so a generic-depth delay wrapper would be dishonest;

longer delays remain explicit source until parameterized delay depth is supported. `std.coding.core` supplies representation-level parity, bit reversal and one-step LFSR operations plus exact dot/convolution and `table_gather` helpers. `table_gather<T,N,IW>` is accepted only when the complete proven `uint<IW>` range fits the source vector. It preserves order but deliberately does not claim bijection: duplicate and omitted elements remain legal. The `CodingLfsrStep<N>` module enforces $N \geq 2$; the compatibility pure function cannot carry a `where` clause because function constraints are not yet syntax.

The language prevents several tempting but invalid "generic" wrappers. A target-independent `reg vec<N,T>` requires an explicit initializer; `default<T>` is not supported. The shipped reusable reorder/ping-pong banks therefore use an explicit `bits<W>` representation boundary. A generic runtime gather retains the conservative element-type range through a table-loaded index, but it does not prove that a table is a mathematical permutation. No backend guesses an initializer or claims a permutation proof. Projects may still use typed ROMs and permutations with known source-generated tables, as the Wi-Fi and FFT sources do.

The separate `std.stream.complex_fixed` profile exposes raw `rv<Complex<...>>` ports. An aggregate `AXIStream` profile cannot be forwarded transparently: hierarchical connections between `AXIStreamOf<Complex<...>>` specializations and member-level child ready/valid ports are not supported by aggregate member binding.

Every shipped `.zh1` file is part of the installed `std.*` source tree. A library import therefore resolves identically from a source checkout and an installed package.

13.3 Target and architecture descriptions

Target libraries use the same safe, hashed `std.*` resolver as bus and math sources. They describe resource capabilities and physical binding locators; they do not add functional primitives. See [target-platform-architecture-description.md](#) for the supported declarations, manual selection flow, and bounded Series-7 implementation.

14 Projects and dependencies

ZLang source imports are logical names. Physical paths, Git URLs, and revisions belong to project metadata, never to `.zh1` source:

```
import acme.dsp.filters
import std.math.complex
```

An import may introduce a source-local qualifier without changing the logical module or dependency identity:

```
import std.math.complex as cx

in sample : cx.Complex<u8>
out total : cx.Complex<u9>
total = cx.complex_sum([sample, sample])
```

The qualifier applies to types, functions, and struct constructors declared by that exact logical module (`cx.Complex { re = ... im = ... }` is valid). The qualified and unqualified spellings identify the same declaration. An aliased import does not expose those declaration names unqualified and does not re-export declarations from a transitive dependency. Aliases are not runtime values or filesystem names; wildcards, member renaming, and re-export remain unsupported.

The built-in `std` namespace keeps its resolver. Other package names are resolved only when the source belongs to a locked project.

14.1 Project manifest

A project is rooted by a versioned `zlang.toml`:

```
schema = 1

[project]
name = "demo"
version = "0.1.0"
source-root = "src"
```

```
[dependencies]
acme = { path = "../acme-dsp" }

[dependencies.bus_models]
git = "https://example.invalid/hardware/bus-models.git"
rev = "0123456789abcdef0123456789abcdef01234567"
```

The schema field above is required user-written project metadata: it tells `zlang lock update` how to interpret the manifest. Use the value shown for the current project format.

Package and module identities contain logical names and content digests, not absolute checkout or cache paths. A file `filters.zhl` directly below package `acme`'s source root is imported as `acme.filters`; nested directories append dotted components.

Path locators are relative to the manifest containing them. Git revisions must be full lowercase 40- or 64-digit hexadecimal object IDs. Branch names, tags, and abbreviated revisions are deliberately rejected because they are mutable.

14.2 Lock update

Dependency resolution and fetching are explicit:

```
zlang lock update --project zlang.toml
```

The command validates the complete transitive graph before publishing a deterministic `zlang.lock`. It records each package manifest, exact source-module index, imports, content digests, and dependency edges. Git content is populated in the project cache during this command. Path dependencies remain at their manifest-relative location and are checked byte-for-byte against the lock.

Resolution rejects:

- dependency cycles and conflicting package/module declarations;
- source-root traversal or symlink escape;
- a package whose declared identity does not match its dependency key;
- missing, added, removed, or modified locked `.zhl` modules;
- modified dependency-resolution fields in dependency manifests (profile-only edits are intentionally outside the resolution identity);
- missing Git cache content or a revision mismatch.

The lock is written only after the entire graph validates. Failed updates leave the previously accepted lock usable.

Scenario	What to do	Without it
Standalone source or built-in <code>std.*</code> imports	Compile directly; no project lock is needed.	No consequence.
Project with local path packages	Run <code>zlang lock update</code> after declaring dependencies and whenever their source or resolution fields change. Keep <code>zlang.lock</code> with the project.	Compilation rejects a missing or stale lock; it never silently uses changed dependency files.
Project with pinned Git packages	Run <code>zlang lock update</code> while Git/network access is available; retain the exact revision and local cache for later offline builds.	Ordinary compilation does not fetch; missing cache or changed revision fails.
Automated build or another machine	Supply the project manifest, its lock, exact path sources and cached Git revisions (or update the lock before the offline build).	A lock file alone does not provide Git source bytes; compilation fails closed.

`zlang lock update` resolves dependencies; it is not a package installer or a step required for every single-file invocation.

Bounded scalar `extern module` implementations may also be declared under `[external-mappings.NAME]` and selected by a profile's `external-mappings = ["NAME"]`. The lock records every HDL source path and SHA-256. Paths are project-relative, must remain inside the project, and may not be symlinks. This is a physical DirectSV input and does not replace the pure ZLang model.

14.3 Offline compilation

Ordinary compilation never fetches dependencies and never updates project metadata:

```
zlang src/top.zhl --project zlang.toml --check
zlang src/top.zhl --project zlang.toml --systemverilog build/top.sv
```

`--project` may name a manifest or its directory. Without it, `zlang` searches the source file's parent directories for `zlang.toml`. If no project is found, single-file and built-in `std.*` compilation work without a project; arbitrary external imports are an explicit error.

Compilation checks the current manifest, lock, every path dependency, and every cached Git dependency before semantic analysis. It is read-only: unavailable or dirty content is a lock mismatch, not an invitation to fetch or rewrite files. Build outputs and ZLang cache directories must remain outside the resolved root, manifest, lock, dependency, and stdlib inputs. The CLI checks the complete physical input set before publication; those host paths are safety guards only and never become semantic or build identity.

14.4 Identity and artifacts

The exact locked dependency closure contributes to build identities and cached formal/synthesis results. Changing dependency content therefore invalidates results even when emitted RTL text happens to remain identical.

In versioned artifact records, the public field `artifact_hash` keeps its narrower meaning: the hash of emitted backend text. The separate build identity combines that text with the logical root module and locked semantic closure. This distinction permits byte-identical RTL to be recognized while preventing evidence from one dependency closure being reused for another.

14.5 Deliberate boundaries

The project model has no registry or semantic-version solver, editable global packages, implicit network access, Git submodules/LFS/subdirectories, wildcard imports, member renaming, or re-export. Implementation profiles are a separate compiler-policy layer and do not change dependency resolution identity. Path dependencies declared by a Git package are also unsupported in this bounded slice; use another pinned Git dependency instead of reaching outside a fetched checkout.

Ordinary compilation remains offline; resolved module contents and their recorded digests are the authoritative dependency identity.

15 Implementation intent and formal verification

ZLang separates hardware meaning from implementation intent. The source expression and its exact types define the result. An implementation request may select among legal realizations, but it cannot change arithmetic, quantization, state, protocol, clock-domain, or externally visible timing semantics.

The implementation flow is conceptually:

```
typed design
  -> legal implementation alternatives
  -> hard-constraint filtering
  -> objective ranking
  -> selected implementation
  -> Direct-SV
```

Optimization is bounded and deterministic. Exact scalar rewrites may simplify pure values, but they do not cross state, storage, protocol, CDC, timing, or quantization boundaries. Ordinary carry-growing arithmetic is not reassocated unless an explicitly supported transformation preserves every intermediate type.

15.1 Semantic and selected designs

Reports distinguish the typed design from the selected implementation. The typed design records exact values, state, timing, domains, and effects. The selected design adds implementation choices such as a pipeline schedule or target resource binding. A selection report is not a new language definition: all legal selections must implement the same source contract.

Estimated and measured evidence are distinct. Resource estimates do not prove physical utilization, and an estimated frequency is not routed timing closure. When a request requires measured evidence, measurements must match the target, implementation, constraints, backend, and tool configuration of the selected design.

15.2 The `implement` form

Use `implement` when an expression has fixed semantics but the compiler may choose among legal implementations:

```
y = implement {
    dot(a, b)
    intent {
        latency <= 4
        ii == 1
        dsp <= 8
        fmax >= 100
        minimize lut
    }
}
```

The source expression remains authoritative. `intent` contains hard constraints and at most one optimization objective. Constraints are never silently relaxed; compilation fails when no legal implementation satisfies them.

Metric	Accepted condition	Meaning
lut, ff, dsp, bram	<=, >=, == nonnegative integer	Estimated resource bound.
latency	<=, >=, == nonnegative integer	Observable sample latency. A positive bound may enable a clocked implementation in a valid clock/reset context.
ii	<=, >=, == positive integer	Initiation interval of a legal implementation.
fmax or fmax_est	<=, >=, == positive MHz value	Frequency constraint subject to the selected evidence policy.
minimize	lut, ff, dsp, bram, latency	Ranking objective. The default is <code>minimize lut</code> .
maximize	fmax or fmax_est	Frequency-ranking objective.

An intent must contain at least one clause. A metric may appear once, and only one objective is allowed. An II constraint describes available implementations; it does not request automatic time-multiplexing. Current scalar alternatives, including the bounded II=1 structural CSA multiplier for exact uniform integer products up to 16 bits per operand, have II=1. Therefore `ii <= 4` may still select II=1 without reducing DSP count.

`pipeline(N) { expression }` is different: it specifies exactly N cycles of observable hardware latency and is not an optimization hint. `choice(...)` contains alternatives authored by the designer. Protocol transform `pipeline(auto, ...)` is the separate bounded ready/valid transformation. Scalar `pipeline(auto)`, `architecture(auto)`, and `explore` are not language forms.

The target planner dispatches operation-specific candidates through typed, target-neutral capability predicates. Generic arithmetic planning does not name Xilinx or SKY130 resources; a target provider may bind a compatible operation to a published hard multiplier, while unsupported shapes remain generic logic. The compiler does not infer CDC, protocol adaptation, general variable-II sharing, quantization movement, or unrestricted retiming from implementation intent.

15.3 Latency, II, and elastic pipelines

A clocked implementation is considered only when the enclosing design has a valid clock/reset context and the intent permits positive latency. Any balancing registers inserted inside the selected implementation are included in its reported latency. Exact module timing and explicit `pipeline(N)` remain authoritative and are not added twice.

The supported ready/valid transform is:

```
input -> output {
  transform pipeline(auto, latency<=3, ii==1, fmax>=100) {
    input.payload.a * input.payload.b
    + input.payload.c * input.payload.d
    + input.payload.e * input.payload.f
    + input.payload.g * input.payload.h
  }
}
```

It accepts one ready/valid input and output in one synchronous domain and a pure supported product-reduction kernel.

Property	Contract
Unstalled latency	One selected fixed value L.
Unstalled II	1.
Capacity	L beats.
Backpressure	Output data and valid remain stable; wall-clock latency may grow.
Pipeline movement	All data registers and valid state advance under one global enable.

Independent per-stage elasticity, stateful kernels, storage, CDC, adapters, and protocol-control capture inside this transform are unsupported.

15.3.1 Capacity-one temporal sharing foundation

The first $II > 1$ candidate is intentionally available only *inside* a typed ready/valid transform, never for an ordinary wire/scalar `implement`. Its current accepted shape is exactly two uniform integer products followed by one integer addition:

```
input -> output {
  transform pipeline(auto) {
    implement {
      input.payload.a * input.payload.b + input.payload.c * input.payload.d
      intent { latency <= 4 ii <= 4 dsp <= 1 minimize dsp }
    }
  }
}
```

II is an accepted-transaction capability, not a decorative ready waveform. After `input.valid && input.ready`, this capacity-one implementation blocks admission while its three scheduled operations run and while its result is pending. On a prior `output.valid && output.ready` retirement it may capture the next input on that same edge; under output backpressure, `valid` and `payload` hold stable and `input.ready` remains low. Latency is the unstalled accepted-input-to-output-valid distance; capacity is separately recorded as one.

The semantic e-graph owns **what** exact value may be computed; the temporal scheduler owns **when**, resource binding owns **where**, and transaction equivalence is separate. A capacity-one transaction miter checks accepted inputs against retired outputs, including same-edge retirement and reload, in bounded formal runs. It is not an unbounded proof and is not yet connected to candidate-selection evidence. Current formal-aware fixed-latency scalar equivalence therefore does not claim proof for this temporal candidate; required formal policy fails closed rather than reusing $II=1$ evidence.

This public-alpha foundation has a deliberately hard boundary.

Supported:

- one ready/valid input and one ready/valid output in the same clock/reset domain;
- the exact integer kernel $a*b + c*d$;
- one shared multiplier, non-interleaved execution, capacity one, latency four, and II four;
- same-edge output retirement and next-input capture under normal ready/valid transfer rules;
- transaction-stream BMC as bounded internal/formal evidence.

Unsupported:

- scalar/fixed-rate $II > 1$ implement regions;
- overlapping transactions, modulo scheduling, arbitrary operation DAGs, or more products;
- memory or CDC sharing, fixed-point temporal sharing, and nested protocols;
- required-formal selection or any claim of unbounded temporal proof.

The checked public example is `examples/temporal_shared_multiply.zhl`.

15.4 Verification goals and contracts

Verification declarations add obligations without changing hardware behavior:

```
assert count_within @ clk {
    count <= DEPTH
}

cover reaches_done @ clk {
    state == State.Done
}

contract fifo_behavior @ clk {
    require legal_input {
        input_length <= 1500
    }
    ensure output_shape {
        !output.valid | output.payload.ok
    }
    cover full {
        queue.full
    }
}
```

`assert` and `ensure` are same-cycle safety obligations. `cover` is bounded reachability and never constrains the design. `require` is an environment-owned precondition local to its contract; requirements in one contract are conjoined and gate that contract's goals. An `ensure` must observe at least one public output. Names are mandatory and unique.

The accepted compatibility forms are:

```
assume bounded @clk disable iff rst {
    (a < 8) & (b < 8)
}

guarantee sum_matches @clk disable iff rst {
    y == a + b
}
```

`assume` is a module-wide environment requirement and `guarantee` is a module-wide assertion. Bodies have type `bit`. Cross-domain expressions, liveness, fairness, arbitrary SVA/SMT, temporal implication, and user-defined formal observation sets are unsupported.

During simulation, verification is sampled after combinational settling and before the edge commit. Active safety failures are design failures. A violated requirement is an environment violation and suppresses dependent conclusions. A cover records its first witness cycle; absence of a simulation witness is not a failure.

Formal execution uses the exact clock/reset contract. When `power_up` unspecified, supported safety and cover jobs handle rising or falling edges, reset polarity, synchronous reset, raw asynchronous reset, and the two-edge

synchronized-release contract. Asynchronous formal execution is single-domain; supported synchronous domains may produce separate jobs. Unsupported observations are reported as unavailable, never guessed from RTL names.

15.5 Running verification

```
zlang design.zhl --verify
zlang design.zhl --verification-bundle build/verify
zlang verify build/verify
```

`--verification-report PATH` with `--verification-format text|json` writes the result. `--verify-require checked|proven` chooses the required safety level. `--formal-depth` and `--formal-timeout` bound execution. `--verification-work-dir DIR`, or `zlang verify --work-dir DIR`, retains generated inputs, solver logs, and VCD traces.

A verification bundle contains immutable, hash-validated design inputs and separate safety/cover jobs. Creating the bundle does not run a solver. `zlang verify` executes those jobs and creates a separate run report. A requested proof runs bounded model checking first; proof begins only after all safety jobs pass the bounded stage. Covers run during the bounded stage and are not rerun for proof.

Safety result meanings are exact:

Result	Meaning
failed	A counterexample was found.
bounded_pass depth=N	No counterexample was found through depth N. This is not proof.
proven	Unbounded proof completed successfully.
unknown	Execution completed without a pass, proof, or counterexample result.
skipped	The goal or route was unavailable or inapplicable. This is not success.
not_run	No proof execution occurred. This is not success.

Cover results are `witnessed cycle=N`, `bounded_unreached depth=N`, `unknown`, or `skipped`. A bounded cover miss is not proof of unreachability and does not by itself fail the command.

Exit status 0 means the requested safety level was satisfied. A safety counterexample returns 1. Incomplete verification, unavailable tools, invalid configuration, or a requested proof with only bounded evidence returns 2.

15.6 Verification boundaries

Fixed-latency $\Pi=1$ implementations may use the supported equivalence route when the required design observations are available. Variable-latency elastic pipelines are outside that relation: their unstalled latency is not a wall-clock latency guarantee under backpressure. A required formal policy rejects unavailable, unknown, skipped, timed-out, or mismatched evidence.

Formal verification does not make an invalid design legal, infer assumptions, authorize CDC, or add new observability. Missing tool support is reported as unavailable. For the accepted coverage matrix, see [Language support matrix](#) and [Known limitations](#).

15.7 Reports

Useful outputs include high-level and selected design reports, implementation/cost/pipeline reports, synthesis results, implementation manifests, and structured verification results. Treat estimated cost separately from synthesis and routed measurements.

16 Optimization model

The scalar optimizer considers only pure, exact-value alternatives supported by the type rules. It does not reorder state, cross timing or quantization boundaries, insert CDC or protocol adapters, or select physical resources by itself. User-declared `equiv` rules must be exact same-cycle value equalities; they are not formal assertions and cannot describe temporal equivalence.

Reports distinguish rewrites, pipeline scheduling, physical mapping, and formal evidence. A rewrite does not imply a pipeline schedule, a structural estimate does not imply physical utilization, and successful timing alignment does not imply proof.

17 Implementation profiles

Implementation profiles keep build policy outside portable .zhl source. They are named tables in `zlang.toml` and are selected explicitly:

```
[profiles.release]
backend = "systemverilog"
backend-mode = "required"
target = "xc7z030ffg676-1"
allowed-transforms = ["pipeline", "dsp", "reduction"]
avoided-transforms = ["reassociate"]
objective = "maximize fmax"
architecture = "Xilinx7SymmetricDSPCascade"
architecture-mode = "preferred"
evidence-policy = "measured_required"
formal-policy = "required_bmc"

[profiles.release.constraints]
latency = { maximum = 8 }
ii = { exact = 1 }
fmax = { minimum = 100 }
dsp = { maximum = 8 }
```

Use a profile with:

```
zlang src/fir.zhl --profile release --systemverilog build/Fir.sv
```

Only the selected profile is applied. An unknown profile, a profile without a project, or an unknown key is an error. Profile constraints cannot weaken or contradict a module's exact timing contract.

A profile may select pinned scalar external-module implementations with `external-mappings = ["vendor-add"]`. Their source files and hashes are fixed by `zlang.lock`. They affect emitted RTL but not the functional source meaning.

17.1 Region selection

`--implementation-policy-report` lists stable identifiers for selectable scalar output regions. A profile may limit policy to exact identifiers with `regions = ["DIGEST", ...]`. Stale, duplicate, unsupported, or non-scalar identifiers are rejected.

17.2 Conflicts

Equivalent policy from source, profile, and command-line options is combined. Conflicting targets, transform sets, objectives, constraints, evidence policies, formal policies, or architectures are diagnosed with their origins. An exact module timing contract may be matched or further bounded, but not changed.

17.3 Reading the selection report

`--backend-implementation-report PATH` explains what Direct-SV emitted:

Status	Meaning
selected	The requested physical implementation was selected and emitted.
generic_fallback	A preferred route was unavailable, so generic RTL was emitted.
unsupported	The requested route is unavailable; required mode fails compilation.
not_requested	No physical route was requested.

A profile cannot introduce protocol, CDC, memory, retiming, simulation, or formal capability that the language implementation does not support.

18 Target platforms

Target selection controls physical implementation; it does not change the functional or timing contract. With no target, or with `--target generic`, ZLang emits generic Direct-SV.

Supported target descriptions include:

- `generic`: technology-neutral RTL;
- `xc7z030ffg676-1`: bounded AMD 7-Series DSP48E1 and memory capabilities;
- Cyclone V descriptions for validation and planning, without Intel primitive emission;
- `sky130-fd-sc-hd`: generic RTL intended for downstream mapping to the SkyWater high-density standard-cell library.

18.1 Manual architecture selection

```
zlang examples/symmetric_fixed_fir.zhl \
--target xc7z030ffg676-1 \
--target-architecture Xilinx7SymmetricDSPCascade \
--target-architecture-mode required \
--systemverilog build/SymmetricFixedFIR.sv \
--implementation-manifest build/SymmetricFixedFIR.manifest.json
```

`required` makes target, shape, width, inventory, dedicated-link, or backend failure an error. `preferred` falls back to generic RTL. `generic`, or omitting target options, preserves generic implementation.

The bounded symmetric-FIR mapping requires eight products, four coefficients reused at mirrored sample positions, and one final `nearest_even/saturate` conversion to `fixed<16,14>`. Runtime-equal coefficients do not establish structural symmetry.

For SF2.10 samples and coefficients, the supported Series-7 cascade uses a 13-bit signed pre-add, 12-bit coefficient input, 25-bit exact product, 27-bit exact accumulation, and a 48-bit cascade path. Quantization occurs once after the complete accumulation; no intermediate is narrowed.

18.2 Target boundaries

The AMD 7-Series path supports the documented bounded DSP48E1 cascade and memory shapes. DSP48E2 and unrestricted placement or graph covering are not supported. Cyclone V primitive emission is not supported. Selecting a target does not claim synthesis, placement, routing, timing closure, or silicon signoff.

For SKY130:

```
zlang design.zhl --top Top --target sky130-fd-sc-hd \
--systemverilog build/Top.sv
```

ZLang emits technology-neutral Direct-SV intended for downstream Liberty-based mapping to `sky130_fd_sc_hd` cells. The ASIC flow must supply the Liberty, LEF, process configuration, SRAMs, PLLs, and project-specific macros. ZLang does not invent hard multipliers, memory macros, PLLs, fixed inventory, timing closure, or signoff results.

19 Target resources

Resource descriptions define legal operations, typed ports and widths, pipeline sites, dedicated connections, memory shapes, clock properties, and inventory. A selected resource must satisfy the exact source types, latency, II, reset, collision, and dedicated-link requirements.

The generic resource set emits ordinary RTL for logic, registers, arithmetic, FIFO storage, RAM, ROM, and clock/control structures. Bounded memory planning may match an advertised shape, replicate 1R1W storage for one-write/multiple-read access, or use the documented register/mux fallback. Automatic banking and clock-resource planning are unsupported.

19.1 DSP48E1

The Series-7 DSP48E1 profile supports the documented signed pre-add, multiply, 48-bit accumulation/cascade path, and legal register sites. The symmetric FIR and ordered signed-product reductions preserve operand order and final fixed-point conversion. Resource-local pipeline stages are eligible only when their latency, II, clock enable, and target capabilities match the source contract.

Intended resource counts are estimates until confirmed by the downstream implementation tool.

19.2 Memories and asynchronous FIFOs

The one-cycle, read-first 1024x36 example can select one `Xilinx7BRAM36SimpleDualPort` when its exact port and collision contract matches. A required binding rejects incompatible write-first behavior, independent-clock ambiguity, or reset policy; preferred mode may use generic RTL.

True-dual-port selection is a distinct RTL shape. A selected Xilinx 2RW route requires preserved contents and emits one physical process per port. Uniform `init VALUE` remains power-up/bitstream initialization under that policy.

The Xilinx asynchronous-FIFO memory bindings apply only to the supported ready/valid `async_fifo` decomposition with independent write/read clocks, one-cycle registered read, and the documented prefetch behavior. They do not select an arbitrary public `async_mem`. Cross-clock read/write collision ordering is not claimed where the target data does not provide it.

19.3 Intel and ASIC status

Cyclone V descriptions cover multiplier modes, accumulation, chain connectivity, pipeline sites, M10K shapes, ALM/FF distinctions, and PLL capabilities. Intel primitive emission is unsupported.

The SKY130 profile records the standard-cell-library boundary but emits generic RTL. Downstream mapping and physical signoff remain the ASIC flow's responsibility.

19.4 Clock resources

ZLang clock domains do not express the frequency/phase relation needed to derive a truthful PLL or MMCM configuration. Automatic PLL/MMCM selection and physical clock primitive emission are therefore unsupported.

20 Target-aware implementation selection

Automatic target-aware selection is limited to the documented symmetric-FIR and ordered signed-product Direct-SV regions on xc7z030ffg676-1.

20.1 User contract

```
result = implement {
    quantize<fixed<16,14>>(acc) {
        round nearest_even
        overflow saturate
    }
    intent { latency <= 8 ii == 1 fmax >= 100 }
}
```

`latency <= N` is a bound; `latency == N` is exact observable latency. `fmax` values are MHz. The compatibility profile spelling throughput normalizes to II.

```
zlang examples/symmetric_fixed_fir_implementation.zhl \
--top SymmetricFixedFIRImplementation \
--target xc7z030ffg676-1 \
--target-evidence-policy measured_required \
--systemverilog build/SymmetricFixedFIRImplementation.sv \
--implementation-manifest build/SymmetricFixedFIRImplementation.json \
--pipeline-report build/SymmetricFixedFIRImplementation.report
```

Source does not name vendor registers or primitives. Generic Direct-SV remains available without a target.

20.2 Evidence and exact latency

Evidence	Establishes
<code>structural_estimate</code>	Compiler estimate; no vendor implementation run.
<code>synthesis_measurement</code>	Post-synthesis result; not routed timing closure.
<code>routed_measurement</code>	Routed result for the matching target, constraints, and implementation.

`measured_required` accepts only compatible routed evidence for an Fmax constraint. Structural latency and II remain exact source/implementation contracts. If an implementation's useful latency is lower than `latency == N`, explicit compensation registers provide the remaining cycles. For `latency <= N`, no compensation is added merely to reach the upper bound.

20.3 Boundaries

Automatic target-aware selection does not perform BRAM banking, PLL/MMCM selection, Intel primitive planning, unrestricted graph covering, II-changing sharing, or automatic fixed-point transformation. Generic Direct-SV remains the fallback when policy permits it.

21 Platform constraints

21.1 Clock constraints

A selected project profile can publish one explicitly named clock period:

```
[profiles.release.platform.clocks.clk]
period-ns = 10.0
```

The clock name must match the selected top's declared clock domain. The period is physical build configuration: it is not inferred from fmax and does not change source timing semantics.

The CLI publishes a constraint only together with exactly one backend artifact. The following command assumes a project-local `top.zhl` and the `release` profile shown above:

```
zlang top.zhl --profile release --systemverilog build/Top.sv \
--constraints-xdc build/Top.xdc --constraints-sdc build/Top.sdc
```

The compiler resolves the RTL port from the selected top's declared clock, never through generated-name guessing. XDC and SDC initially contain only:

```
create_clock -name clk -period 10 [get_ports {clk}]
```

Each generated constraint artifact retains its backend hash, selected implementation identity, clock edge, and complete reset mode/polarity/release-cycle/power-up metadata. For a non-default contract, publication also requires an exact physical-domain record in the implementation artifact; stale or mismatched reset semantics are rejected. Whole-build manifests publish the file as a backend companion and include the platform profile and constraint identities. XDC/SDC output is deterministic and atomically published without following symlinks.

No reset false path is emitted: asynchronous-reset recovery/removal timing must not be hidden. This constraint publication supports only the exact single-clock profile shown here. Multiple clock domains are supported in language semantics and Direct-SV emission, but generated clocks, multi-clock XDC/SDC publication, pin/package/IO properties, input/output delays, CDC constraints, and formal properties are not emitted by this facility.

22 Direct-SV

Direct-SV is generated from the fully typed design. Standard-bus components use the same aggregate endpoints, hierarchy, state, rules, and connections as user modules; module names do not trigger special bus behavior.

Public top ports and source register names follow the documented ABI. Generated private helpers are deterministic and collision-safe for reproducible artifacts, but their spelling is not a public integration contract. Designs must not connect to private generated names.

Formal checks that observe rule firing use the same effective reset and polarity as the generated state, including two-edge synchronized release and conditioned child reset. Rising/falling edges, both polarities, and nested hierarchy are supported. Checking many independent rule guards remains bounded.

Fixed-point ports and expressions use the canonical scaled-integer types behind `fixed/ufixed`, concise `SF/UF`, and `_sat` formats. Same-scale target narrowing and explicit rescaling preserve the declared conversion; Direct-SV never infers wrap, saturation, or rounding from source spelling.

22.1 Supported composition subset

Direct-SV emits the language/backend intersection listed in the [support matrix](#). Its RTL-specific contracts are:

- exact typed widths, signedness, fixed-point conversions, packing order, and declared pipeline latency;
- registers, atomic rules, FSMs, FIFO/memory behavior, and every explicit clock/reset domain use their documented state and reset semantics;
- typed hierarchy remains physical hierarchy, including the bounded instance-array and runtime output-projection forms;
- ready/valid, credit, request/response, packet, CSR, and supported CDC forms lower from their declared protocol and domain contracts without name-based inference;
- the public top ABI exposes struct fields and tuple items as named leaves, vectors as multidimensional packed arrays, and tagged unions as named packed tag/payload structures; compact packed aliases remain private to generated RTL; and
- source-authored standard buses remain ordinary typed hierarchy rather than backend-specific transaction primitives.

Unsupported combinations fail before an RTL artifact is published. The support matrix is the authoritative exhaustive inventory; this section defines how supported designs appear in SystemVerilog.

SystemVerilog reserved words are deterministically prefixed with `zlang_`. The collision check runs after that physical-name mapping, so two distinct semantic leaves can never silently become one RTL port. Inline-boundary helper signals are allocated outside the public namespace and remain deterministic. Packed dimensions use conventional descending ranges. ZLang element zero maps directly to physical packed index zero and the least-significant helper slice. The generated artifact has one unconditional top definition and is accepted by both Yosys and Verilator without backend preprocessor branches. Unresolved locals are eliminated before emission. Unsupported typed designs are diagnosed, and no artifact is published after failed emission.

Ordered comparisons preserve typed signedness explicitly: both operands of `<`, `<=`, `>`, and `>=` are rendered through `$signed` or `$unsigned` according to the declared operand type, independent of whether an operand is a port, local, projection, or register. Equality and inequality remain raw bit comparisons.

Use the stable CLI option:

```
zlang examples/simple_dma.zhl --top SimpleDMA \
  --systemverilog build/SimpleDMA.sv
verilator --lint-only --top-module SimpleDMA build/SimpleDMA.sv
```

`--systemverilog` is the sole Direct-SV output option. It writes only the requested SV file and leaves stdout empty. A bare `zlang SOURCE` invocation is rejected because every compile action or artifact sink must be explicit.

Use `--verbose` when an explicit success confirmation is useful:

```
zlang design.zhl --systemverilog design.sv --verbose
```

This keeps stdout artifact-safe and reports the selected top and written paths on stderr. To validate parsing, top selection, and semantic analysis without publishing any backend artifact, use:

```
zlang design.zhl --check
```

`--check` prints a one-line success result and returns zero. Without `--top` it checks every module declared in the source file, preventing an invalid dormant module from being hidden by default-top selection. With `--top`,

it checks only that selected elaboration root. Syntax, semantic, and top-selection failures retain their normal nonzero status and diagnostic. As a check-only mode, it cannot be combined with artifact output options.

22.2 Simulation-only architectural state access

A Direct-SV build may publish a separate Verilator VPI companion without adding ports or changing the production RTL:

```
zlang design.zhl \
  --systemverilog build/design.sv \
  --simulation-state-bundle build/state-access
```

The companion manifest covers the selected typed hierarchy and is bound to the exact emitted artifact hash, build identity, state shapes, and physical state locations. Publication re-hashes the .sv file, so a stale or modified RTL file cannot receive an access manifest. The generated C++ header requires Verilator `--vpi --public-flat -rw` and validates its complete state allow-list before access.

The bounded surface includes bit-packable user registers (including vector registers), writable-memory cells, and the persistent read-result latch of a one-cycle memory. Values up to 64 bits have convenience methods; arbitrary-width scalars and elements use exact 32-bit least-significant-word-first arrays. Vector element zero retains the canonical least-significant packed position. Native simulation uses the same state catalog and keeps one state object per physical child instance.

22.3 Formal boundary

The executable Direct-SV formal route supports register transitions and rule priority, ready/valid stability and FIFO accounting, CSR W1C behavior, and request/response acceptance. A failing property reports counterexample metadata when the solver provides it.

The verification-bundle flow also accepts Direct-SV designs with initialized ROM. Exact memory images are published below `implementation/companions/` and listed as hash-validated inputs for every SBY job that consumes them. Generated solver configuration, logs, and VCDs are retained outside the immutable bundle; the run report maps sampled physical VCD signals back to their ZLang semantic signal identities for witnesses and counterexamples.

Direct-SV is the executable bundle route. If its formal emission cannot bind a required observation, the goal is explicitly skipped or rejected according to policy. Unsupported aggregate/protocol shapes report a non-executable reason; no generated-name reconstruction or alternate backend substitution is used.

`RegBusCSRTarget` stores response data and holds `response.valid` until `response.transfer`. Source-authored AXI-Lite and APB frontends can therefore complete write transactions through the typed `RegBus` boundary.

The implementation artifact publishes exact state locations only for signals actually present in emitted RTL. Formal checking can observe supported request acceptance, outstanding-transaction accounting, directional buffering, response consumption, register state, FIFO state, CSR state, and action firing when those signals are available in the generated artifact. Hidden or unsupported state is not guessed, and the affected property reports `skipped` rather than fabricating a proof.

22.4 Explicitly unsupported

Globally controlled storage combined with user state, out-of-order or nested request/response arrays, additional protocols on a request/response array child, arbitrary nested storage/CSR/aggregate protocol hierarchy, CDC arrays, and runtime-selected inputs, protocols, or actions remain fail-closed. The backend never silently emits only one element or reconstructs an indexed connection from an RTL name.

Sequential ready/valid modules and role-qualified standalone in-order `request_response` requester/responder modules use the same typed clock/reset, ownership, and accounting semantics as their hierarchical forms. A standalone endpoint owns a local unbuffered ledger: request transfer increments it, response transfer decrements it, and reset starts a new empty epoch. Stateful generic child templates that require parent specialization are checked through their concrete parents.

22.5 Examples

The streaming FFT and IEEE 802.11a transmitter examples combine typed hierarchy, ROMs, fixed-point datapaths, protocols, and state through the same Direct-SV path. Child/template modules require a concrete

specialization or parent. The [support matrix](#), rather than the example list, defines the supported boundary.

Generated RTL uses exact packed widths and single-driver combinational assignments. Procedural combinational blocks are used where defaults or branching require them; strict lint treats width, latch, driver and structural warnings as failures. A target binding is claimed only when the selected physical capability proves its clock, latency, collision and reset contract.

23 Editor and tooling integration

The read-only `zlang.tooling` Python API supports editors and build tools that need stable compiler results without importing parser or semantic implementation modules. It provides source checks, diagnostics, document symbols, hover, definition, references, rename, completion, signature help, semantic tokens, project indexing, and dependency information.

Queries use the supplied source snapshot. They do not emit RTL, run synthesis, or execute formal tools. A source error is returned as a structured diagnostic; a workspace or environment failure that prevents a trustworthy answer raises `ToolingError`.

Navigation and rename use resolved declaration identity, not identifier spelling. Completion uses the declarations visible in the semantic scope at the requested position. Signature help describes the callable actually selected by the compiler. The API never returns parser trees, semantic environments, or backend objects.

23.1 Diagnostic edits

A diagnostic may include one or more explicit machine-applicable fixes. Each fix has a title and an atomic group of exact source edits. These edits are separate from human-readable suggestions: tools must not derive replacements from a diagnostic message, code, or prose suggestion.

The machine-applicable fix removes the later of two imports with the same logical path and alias. Every edit must match the current source snapshot; stale, ambiguous, broad-span, and cross-file edits are omitted.

23.2 Query behavior

- `document_symbols(source_text)` returns the declaration hierarchy retained by parsing. Invalid or incomplete source returns no symbols.
- `hover_at(...)` returns canonical type, width, signedness, direction, and callable signature information where available.
- `definition_at(...)` resolves local and imported values, functions, named types, enum members, registers, and module-instance targets when an exact declaration location is available.
- `references_at(..., include_declaration)` returns exact resolved uses, optionally including the declaration, in deterministic source order.
- `rename_at(..., new_name)` validates the identifier, edits only exact name spans, includes the declaration, and rejects collisions or retargeted uses. The supported rename categories are local ports, immutable values, functions, and function parameters.
- `completion_at(...)` returns visible ports, values, parameters, ordinary and generic functions, and functions imported by the locked project. It does not provide keyword, snippet, or auto-import completion.
- `signature_help_at(...)` returns the resolved callable signature and active parameter for ordinary, generic, and imported calls.
- `semantic_tokens(...)` classifies exact occurrences of functions, parameters, ports, and immutable values. Declarations are marked separately from uses.

Malformed or incomplete input returns an empty result where the requested semantic fact cannot be established. There is no textual or regular-expression fallback.

24 Language Server Protocol and VS Code

Start the Community language server with:

```
zlang lsp
```

The VS Code extension launches this subcommand from the configured `zlang` executable. The server uses standard Content-Length framed JSON-RPC over stdio, accepts local `file://` URIs for `.zhl` files, and keeps open document text in memory.

24.1 Supported requests and notifications

- `initialize`, `initialized`, `shutdown`, and `exit`;
- `textDocument/didOpen`, `didChange`, and `didClose` with full-text synchronization;
- `textDocument/publishDiagnostics`;
- `textDocument/documentSymbol`;
- `textDocument/hover`;
- `textDocument/definition`;
- `textDocument/references`;
- `textDocument/rename`;
- `textDocument/completion`;
- `textDocument/signatureHelp`;
- `textDocument/semanticTokens/full`;
- `textDocument/codeAction` for compiler-provided quick fixes.

Positions and ranges use the LSP UTF-16 convention. The server uses the exact in-memory text of the current document. Saved dependencies come from the locked project snapshot; unsaved overlays for other files are not combined with that snapshot.

Definition and References may cross files only when the locked project supplies an exact declaration identity and source location. References optionally include the declaration. Rename always includes the declaration, edits exact identifier spans, and rechecks the proposed source before returning a workspace edit.

Completion is semantic rather than lexical. Malformed input or an unsupported cursor context produces an empty list rather than a guessed global symbol list. Signature help chooses the innermost resolved call and derives the active parameter from argument spans rather than comma counting.

Semantic tokens use a fixed legend and full-document relative encoding. The supported token roles are functions, parameters, ports, and immutable values; the server does not use semantic tokens for hardware-resource visualization.

Code actions recheck the current document and expose only explicit machine-applicable `quickfix` edits. Stale diagnostics and prose-only suggestions produce no action.

24.2 Cache configuration

Unchanged saved project snapshots may reuse exact semantic and symbol results. Any change to editor text, selected module, manifest, lock, or used dependency invalidates the affected result. Unsaved buffers remain in memory. Set `ZLANG_LSP_SYMBOL_CACHE=memory` to disable disk persistence or `off` to disable symbol shards; `persistent` is the default. Corrupt or unreadable entries are ignored and recomputed.

24.3 Not implemented

`codeAction/resolve`, `fix-all/source/refactor` actions, semantic-token range/delta requests, workspace symbols, generated-RTL navigation, formal/synthesis commands, and unlisted LSP features are unsupported.

25 Structured diagnostics

Compiler errors retain a human-readable message and may also provide a stable code, severity, primary source range, notes, and suggested fixes. Source ranges are half-open. Logical source identity and a content digest are included when the compiler has a complete saved-source origin.

Text output is the default:

```
zlang design.zhl --check
```

Machine consumers can request one deterministic JSON object:

```
zlang design.zhl --check --diagnostic-format json
```

The JSON object contains `schema`, `severity`, `code`, `message`, `primary`, `notes`, and `fixes`. The values in `fixes` are human-readable suggestions, not executable edits. Executable edits use the separate machine-fix contract described above.

A generated source map may attribute a Verilator, Yosys, or vendor diagnostic to ZLang source only when the generated artifact hash matches and exactly one entry covers the reported line. Otherwise the original external-tool diagnostic is preserved without guessed attribution.

26 Generated RTL source maps

Direct-SV can emit a deterministic JSON sidecar that connects generated line ranges to exact ZLang source origins:

```
zlang design.zhl --systemverilog Design.sv \
  --source-map Design.sv.zmap.json
```

The map records the generated artifact hash. A consumer must verify that hash before using any attribution. Changed files, malformed locations, ambiguous entries, and unmapped lines remain unattributed.

26.1 Supported coverage

Source maps cover unique top-level output assignments and the final output assignment of an eligible pipeline when exact source origin is retained. Module declarations, internal state, helper logic, protocol bridges, and most hierarchy are not mapped. Ranges identify generated lines, not columns.

No mapping is inferred from similar source and RTL names or from proximity. Zero matching entries means “unmapped”; multiple distinct source origins are ambiguous. Source columns use compiler code-point positions; an LSP client converts them to UTF-16 only at the protocol boundary.

27 Generated artifact bundles

A generated-navigation bundle packages one already-generated Direct-SV file, its source map, its backend manifest, and the producing source identities into one relocatable directory. It validates artifact lineage and source freshness; it does not add an LSP navigation method.

Publish a bundle during compilation:

```
zlang design.zhl --top Design \
  --systemverilog build/Design.sv \
  --source-map build/Design.source-map.json \
  --build-manifest build/Design.build.json \
  --generated-navigation-bundle build/Design.navigation
```

The bundle contains:

```
manifest.json
generated/design.sv
generated/source-map.json
manifest/backend-artifact.json
```

All paths are relative to the bundle root. Loading verifies supported format, safe path containment, regular files, sizes, hashes, canonical manifests, generated bytes, module/backend identity, source-map lineage, and complete source identities. Missing, stale, tampered, path-traversing, or symlinked contents are rejected. Loading reads metadata only; it does not compile source, emit RTL, or execute tools.

A loaded bundle reports `match`, `stale`, or `unknown_source` for a caller-supplied source text or digest. Bundles do not embed source contents, discover workspaces, manage unsaved overlays, or extend the bounded source-map coverage above.

28 Build manifests and evidence

`--build-manifest PATH` writes a deterministic record joining the source and locked dependency identities, selected implementation policy, Direct-SV products, companion files, actual external-tool executions, reports, and verification evidence.

The manifest distinguishes the typed design identity from the selected implementation identity and from generated artifact hashes. This prevents a physical implementation plan from being mistaken for source semantics. Successful backend states publish their products; unsupported, failed, and unrequested states cannot appear as successful artifacts.

Published files use normalized relative paths with SHA-256 and size. Absolute paths, parent traversal, duplicate paths, and symlink escapes are rejected. Initialized ROM images remain separate content-addressed companions to the RTL that reads them.

28.1 Evidence meanings

Status	Meaning
<code>typed_legal</code>	Semantic analysis accepted the module. This is not timing evidence or proof.
<code>timing_validated</code>	Public scalar-output timing matches the exact module contract. This is not proof.
<code>bounded_pass</code>	BMC found no counterexample through the recorded depth. This is never unbounded proof.
<code>proven</code>	An unbounded proof completed successfully.
<code>failed</code>	An executed check found a failure.
<code>unknown</code>	Execution completed without a conclusive result.
<code>skipped</code>	The route was unavailable or inapplicable. This is not success.
<code>not_run</code>	No proof execution occurred. This is not success.

Evidence is accepted only from a matching execution and artifact. A generated harness, installed solver, cached record, or structural estimate is not proof.

28.2 Bundle generation and proof execution

`--verification-bundle` creates immutable verification inputs; it does not run a solver. `zlang verify` or `zlang --verify` creates results only after real execution. Solver, engine, depth, timeout, logs, tool versions, traces, and counterexamples belong to the run report.

Each job belongs to one clock/reset domain. Supported multiple synchronous domains may produce separate jobs; this does not create a cross-domain equivalence relation. A proof request begins with bounded checking. Covers run once during the bounded stage and do not block unrelated safety proof, although an unwitnessed feasibility cover makes its dependent safety result inconclusive.

28.3 Manifest and report commands

```
zlang design.zhl --systemverilog build/design.sv \
  --evidence-report build/evidence.json \
  --evidence-format json \
  --build-manifest build/zlang-build.json
```

`--evidence-format text|json` selects the report representation. `--evidence-report` writes that report, and `--build-manifest` publishes the build record after all requested outputs have been written and validated. A build manifest requires a real backend product; check-only and report-only commands do not fabricate one.

Build identity includes source/dependency content, typed and selected design identities, normalized implementation policy, backend products, executed tools, reports, and stable evidence. Host paths, timestamps, wall time, and cache hit order do not change it.

28.4 Limitations

Generated source mapping is line-only and intentionally sparse. Artifact bundles do not provide source-to-generated queries. Build manifests record executed work but do not claim implementation timing closure or proof beyond the exact evidence statuses above.

29 Source and product identity

The public product name is **ZLang HDL**. Its distribution and repository are `zlang-hdl`; the compiler command is `zlang`, the source suffix is `.zh1`, the VS Code language ID is `zlang-hdl`, and the MIME type is `text/x-zlang-hdl`.

The `.z1` suffix is not a compatibility alias. Physical compiler inputs using it fail with diagnostic `ZL-SOURCE-EXTENSION`; use `.zh1`.

Logical imports do not contain a source suffix. For example, `import std.math.fixed` resolves through the built-in `std` namespace, while project and locked-package imports use their logical module identities.

Unsupported dependency-lock formats are rejected; regenerate them with `zlang lock update`. Cached source, build, and proof results are reused only when dependency and source identities still match.

The short prose name **ZLang** remains valid after the product has been introduced. The Python package `zlang`, `zlang.toml`, `zlang.lock`, the `.zlang/` state directory, environment variables prefixed `ZLANG_`, generated HDL identifiers, and the TextMate scope `source.zlang` are intentional product identities.

30 Language support matrix

Canonical implementation-selection markers: `implement`, `choice`.

Implementation selection uses the canonical `implement` form for compiler-discovered candidates and `choice` for user-supplied alternatives. The retired scalar `pipeline(auto)`, `architecture(auto)`, and `explore` spellings are not grammar productions; only protocol transform `pipeline(auto, ...)` remains source syntax.

The following table is the public language/backend support view. `supported` means the feature is available under its stated type and domain rules; `bounded` means it is supported only within explicit limits in this reference; `experimental` means it is available in the alpha but is not yet a stable contract; and `unsupported` or `not supported` means the form is rejected. `Simulation` means support by the native simulation contract. Native execution rejects unsupported plans explicitly; see [Native simulation](#). Formal entries name the specific eligible relations rather than promising arbitrary proof.

30.1 Language and execution

Capability	Language	Simulation	Direct-SV
<code>scalar-datapath</code>	supported	supported	supported
<code>exact-literals-and-packed-constants</code>	supported	supported	supported
<code>fixed-point</code>	supported	supported	supported
<code>aggregates</code>	supported	supported	supported
<code>characters-strings-tuples</code>	bounded	supported	supported
<code>tagged-unions</code>	bounded	supported	supported
<code>functional-datapath</code>	supported	supported	supported
<code>compile-time-generation</code>	supported	not applicable	supported after specialization
<code>concise-exact-lowering</code>	supported	supported	supported
<code>typed-static-parameters</code>	bounded	not applicable	supported after specialization
<code>generic-rom-and-table-gather</code>	bounded	supported	supported
<code>sequential-state</code>	supported	supported	supported
<code>physical-clock-reset</code>	bounded	supported	supported
<code>multi-clock-stateful-logic</code>	bounded	supported	supported
<code>encoded-enums-and-fsm</code>	supported	supported	supported
<code>vector-state-update</code>	bounded	supported	supported
<code>fifo-storage</code>	supported	supported	supported

Capability	Language	Simulation	Direct-SV
writable-memory	bounded	supported	supported
ready-valid	supported	supported	supported
credit	supported	supported	supported
request-response	supported	supported	supported
aggregate-protocols	bounded	supported	supported
ahb-lite-stdlib	bounded	supported	supported
cdc	bounded	supported	supported
csr	bounded	supported	supported
contracts	supported	supported	artifact generation
exploration	bounded	not applicable	selected candidates
elastic-ready-valid-pipeline	bounded	supported	supported
combinational-instance-arrays	bounded	supported	supported
runtime-instance-output-projection	bounded	supported	supported
sequential-instance-arrays	bounded	supported	supported
storage-instance-arrays	bounded	supported	supported
ready-valid-instance-arrays	bounded	supported	supported
request-response-instance-arrays	bounded	supported	supported

30.2 Formal coverage

Capability	Formal coverage
scalar-datapath	same-cycle equivalence where eligible
exact-literals-and-packed-constants	same-cycle equivalence where eligible
fixed-point	fixed-point equivalence within supported widths and forms
aggregates	value equivalence where all required fields are observable
characters-strings-tuples	scalar and packed-value equivalence where eligible
tagged-unions	no dedicated tagged-union proof support
functional-datapath	scalar and fixed-point equivalence where eligible
compile-time-generation	compile-time only; formal execution not applicable
concise-exact-lowering	same formal coverage as the equivalent explicit source form
typed-static-parameters	not a runtime formal target; specialized values participate in the elaborated design being checked
generic-rom-and-table-gather	storage safety checks only; no dedicated table-gather proof
sequential-state	register and rule safety checks where the required state and events are observable
physical-clock-reset	reset-aware properties for supported edge, polarity, assertion, and release modes when <code>power_up</code> is unspecified
multi-clock-stateful-logic	source goals are checked in their declared domains; automatic safety checks are bounded
encoded-enums-and-fsm	no dedicated enum or FSM proof support
vector-state-update	register safety where the selected element state is observable
fifo-storage	bounded FIFO safety checks
writable-memory	no dedicated memory-content proof support
ready-valid	payload-stability and transfer safety where observable
credit	sender and receiver credit safety when the exact counter is observable
request-response	outstanding-transaction and directional-buffer safety where observable
aggregate-protocols	no general aggregate-protocol proof support
ahb-lite-stdlib	register-state and ready/valid safety where signals are observable
cdc	no CDC correctness proof
csr	CSR state and access safety checks where observable
contracts	safety checking and bounded cover execution when supported
exploration	candidate equivalence can gate required selection when a supported equivalence check is available
elastic-ready-valid-pipeline	ready/valid stability safety only; end-to-end equivalence is not supported
combinational-instance-arrays	hierarchical equivalence is not supported
runtime-instance-output-projection	hierarchical equivalence is not supported
sequential-instance-arrays	bounded safety for exposed child state and events; no general recursive hierarchy proof
storage-instance-arrays	no formal access to unexposed internal storage contents
ready-valid-instance-arrays	ready/valid safety where the required signals are observable
request-response-instance-arrays	request/response safety where the required signals are observable

31 Known limitations

ZLang 0.1.0a19 is an experimental alpha release. A design outside the documented language/backend intersection is rejected; RTL is never published after silently dropping a source construct.

31.1 Language and backend boundaries

- ZLang has no runtime procedural `if`, mutable local variables, implicit numeric casts, implicit fixed-point rescaling, or general HLS scheduler.

- Runtime-selected instance inputs/protocols, general cross-module atomic scheduling, general drop-in AXI4 endpoint compliance, automatic CDC insertion, and arbitrary stateful elastic pipelines are outside the alpha contract.
- Stateful objects inside one module may belong to independent explicit clock domains. A bounded `async_mem` is the sole storage exception: its one writer and one registered reader have distinct owners and form an explicit semantic boundary. Asynchronous 2RW memory, mixed widths, automatic banking, cross-clock atomic rules, derived/gated clocks, and target-aware scheduling across a CDC boundary are not implemented. Ordinary globally controlled memory cannot be combined with unrelated user register/rule state in the same module; use hierarchy for that composition.
- The Direct-SV production intersection is authoritative. Unsupported combinations must produce a structured diagnostic rather than partial RTL.
- The Python API is provisional. The command-line interface and versioned artifact/lock/bundle schemas are the intended integration surfaces.
- Simulation-only state access is limited to one exact clock/reset domain. This generic Direct-SV and Verilator facility does not expose backend-created FIFO, CSR, protocol, CDC, or target-mapped state and must not be confused with synthesizable memory initialization.

31.2 Verification boundaries

The machine-readable capability registry and [syntax support matrix](#) are the detailed authorities for individual constructs. Report any accepted design that emits invalid RTL as a correctness defect.

A ZLang HDL concise source-authoring reference

This page is a concise guide to executable ZLang HDL source. ZLang source defines semantics; generated SystemVerilog is an implementation artifact, not a second language specification.

For an unfamiliar construct, consult the [syntax support matrix](#) and [known limitations](#). For exact width tables and backend limits, follow the linked topic guide.

A.1 Non-negotiable rules

- Source files use .zhl. The old .zl suffix is rejected.
- Run `zlang SOURCE --check`; highlighting is lexical and is not validation.
- Native simulation requires the matching native-simulation package.
- Hardware assignments are concurrent. `=` drives the current cycle and `<-` schedules next state at the active clock edge.
- Assignment types are exact. There is no implicit resize, signedness change, fixed-point rescale, protocol adapter, or clock-domain crossing.
- Do not invent `let`, `const`, `var`, `return`, semicolons, C-style input/output, or runtime procedural `if`.
- `if` is compile-time elaboration only. Use `?:`, `mux`, or `switch` for pure runtime values; use `when/else when/else`, `priority`, or `fsm` for state actions.
- Scalar `pipeline(auto)`, `architecture(auto)`, and `explore` are retired. Compiler-selected scalar implementation uses `implement`.
- `pipeline(N) { expression }` is exact N-cycle hardware, not an optimization request. `Protocol transform pipeline(auto, ...)` is a separate bounded ready/valid construct and remains supported.
- Unsupported combinations are diagnosed explicitly. Never work around a diagnostic by guessing generated signal names.

A.2 Smallest valid modules

Combinational logic:

```
module Add {
    in a, b : u8
    out y : u9 = a + b
}
```

Sequential logic:

```
module Accumulator {
    clock clk reset rst
    in enable : bit
    in value : u8
    out total : u9

    reg state : u9 = 0
    update: when enable {
        state <- truncate<9>(state + extend<9>(value))
    }
    total = state
}
```

All operands are read from one pre-edge snapshot. The effects selected by one action commit atomically; reset has priority. Multiple conflicting actions require an explicit priority relation or are rejected.

Multiple domains use the same declarations with one @clock owner:

```
module DualClock {
    clock a reset ra @a
    clock b reset rb @b
    reg left : u32 @a = 0
    reg right : u32 @b = 0
    rule TickLeft @a when 1 { left <- truncate<32>(left + 1) }
    rule TickRight @b when 1 { right <- truncate<32>(right + 2) }
}
```

An omitted owner is accepted only when exactly one domain can be inferred. Dynamic provenance follows through local expressions; an explicit `sync_level`, `pulse_toggle`, `handshake`, or `async_fifo(N)` is the only way to cross domains.

A.3 Types, literals, and exact conversions

Common scalar types are `bit`, `u8`, `s16`, `uint<N>`, `sint<N>`, and `bits<N>`. `char` is canonical `u8`; `string<N>` is canonical `vec<N,u8>`. Fixed point uses `fixed<W,F>`, `ufixed<W,F>`, or concise forms such as `SF8.8` and `SF_Sat8.8`.

- Unsuffixed integer literals use minimum exact width unless they occur directly at a concrete typed boundary.
- `a + b` and `a - b` follow the documented exact result-width rules; multiplication returns the full product width. Shifts preserve left width.
- Use `extend<N>` and `truncate<N>` for width changes.
- Use `quantize<T>` for explicit fixed-point rounding/overflow. Do not move a quantization boundary into products or reductions.
- Compile-time `pi`, `sin`, `cos`, `log2`, `log`, `exp`, and `sqrt` generate constants only; non-integral results require explicit `quantize<T>`.
- Use `bitcast<T>` only for equal-width representation changes. `pack/unpack` are supported low-level compatibility forms.
- `concat(a,b,...)` places the first operand at the MSB. Destination context never changes operand widths.
- Packed bit index zero is the LSB. `x[MSB:LSB]` uses an inclusive static slice; vector ranges `values[first..past_last]` are half-open.
- Indexed aggregates are packed LSB-first: `vec/string` element zero and tuple `item0` occupy the least-significant component recursively. First-declared struct fields and tagged-union tags remain at the MSB.

See [types and numerics](#) before changing a width or a fixed-point type.

A.4 Aggregates and functions

```
struct Pair<type T> { left : T right : T }

fn duplicate_sum<type T>(a : T, b : T) {
  exact = a + b
  Pair { left = exact right = exact }
}
```

Function bodies are pure and combinational. Bare body bindings are immutable, inferred, declaration-ordered locals; the final expression is the result. Return types may be explicit or inferred. Functions cannot capture module ports or contain state, protocols, delays, or pipelines.

Supported aggregates include `vec<N,T>`, nominal `struct`, nominal `enum`, tagged union, and structural tuples `(T,U)`. Prefer inferred locals when the type is exact and obvious. Keep explicit types at public ports, state, raw/enum boundaries, narrowing, and fixed-point quantization.

Generics are monomorphized. Type/value parameters and exact compile-time constraints are source declarations, not runtime hardware:

```
module Queue<type T,D=4> where D >= 2 && is_power_of_two(D) {
  // ...
}
```

Imports are logical. For example `import std.bus.reg` resolves through the built-in `std` namespace; `import std.math.complex as cx` adds a source-local qualifier but does not create a runtime namespace.

A.5 State, hierarchy, and protocols

- Declare one default synchronous domain with `clock clk reset rst`.
- `async reset arst @clk` means asynchronous assertion and fixed two-edge synchronized release. Do not emulate reset modes in user logic.
- Add `release externally_synchronized` only when deassertion is already synchronized outside; no internal 2FF is emitted.
- `reg q:T = VALUE` resets to `VALUE`; `reg q:T` has no reset branch.
- `reg`, `fifo`, `memory`, and `rom` retain explicit typed state/storage semantics. Memory latency, collision, mask, and reset policies are not inferred.
- A concise child declaration is `child : ChildModule`; `inst child : ChildModule` remains accepted. Bind only typed ports/endpoints.

- `rv<T>`, `credit<T,N>`, and `request_response` properties such as `.transfer` are typed protocol status values, not ordinary struct fields.
- Use explicit `connect/source -> sink`, adapters, and CDC crossings. The compiler never inserts these silently.
- Standard buses are source-authored profiles under `std.bus.*`: use `axi_lite`, `apb`, or `ahb_lite` with their provided RegBus bridges; use `axi_burst` for the bounded no-ID/single-outstanding profile; use `axi4` only for its documented five-channel bounded model. No import implies automatic adaptation or general endpoint compliance.
- Top-level structs and tuples become recursively named leaf ports; vectors use multidimensional packed arrays in the production Direct-SV ABI.
- Top-level tagged unions are valid inputs and outputs. Direct-SV exposes their exact tag and payload through a named packed structure; external producers must drive a declared tag code.

Read [sequential state/storage](#) and [hierarchy/protocols](#) before composing stateful children. The exact bus inventory and runnable examples are in [standard buses](#).

A.6 Functional datapath and implementation intent

`generate`, `map`, `reduce`, `sum`, and `dot` are statically bounded functional hardware expressions. They are not runtime loops.

Use exactly one `implement` region when the value is fixed and the compiler may choose among legal implementations:

```
y = implement {
  dot(samples, coefficients)
  intent {
    latency <= 4
    ii == 1
    dsp <= 8
    fmax >= 100
    minimize lut
  }
}
```

The expression is the hardware meaning. `intent` contains hard constraints and one supported objective; it does not name optimization passes. Positive-latency alternatives require a valid clock/reset context. `choice` is reserved for user-authored equivalent alternatives. Pure scalar optimization never schedules pipelines or state.

The only current `II>1` implementation is the bounded ready/valid `a*b + c*d` transform: one shared multiplier, capacity one, unstalled latency four, and `II` four with same-edge retire/reload. It is not available to scalar `implement`, does not overlap transactions, and has only bounded transaction-stream evidence; required-formal selection remains fail-closed.

See [optimization and formal](#) before adding or changing an implementation policy.

A.7 Verification

```
contract arithmetic @ clk {
  require legal_inputs { (a < 8) & (b < 8) }
  ensure exact_output { y == a + b }
  cover maximum { y == 14 }
}
```

Named `assert/ensure` are same-cycle safety goals. `require` is an environment-owned precondition scoped to its contract. `cover` is bounded reachability; a missed cover is not an unreachable proof. BMC produces `bounded_pass`, never `proven`. Verification syntax does not legalize invalid hardware or feed optimizer range inference.

Liveness, arbitrary SVA/SMT, temporal sequences, and user-defined formal observation sets are not supported. See the [formal examples](#) for the available flows.

A.8 Commands and completion check

```
zlang source.zhl --check
zlang source.zhl --top Top --systemverilog build/Top.sv
zlang source.zhl --top Top --target sky130-fd-sc-hd --systemverilog build/Top.sv
zlang source.zhl --top Top --verify
zlang sim source.zhl --top Top --engine native --clock clk --cycles 100
zlang verify build/verify --mode bmc --depth 20 --work-dir build/verify-work
```

```
zlang lock update --project zlang.toml  
zlang lsp
```